Facultad de Ingeniería
Universidad Nacional de Mar del Plata

Universidad Nacional de Mar del Plata
de Mar del Plata

Universidad Nacional de Mar del Plata

Facultad de Ingeniería

Trabajo final

Desarrollo de un Analizador Vectorial de Redes para medición no invasiva de glucemia

---

Victoria Carla Torres



Ingeniería Electrónica
Departamento de Ingeniería Electrónica y Computación
Septiembre 2026

Agradecimientos

Índice

Agradecimientos **1**

Siglas y abreviaturas **4**

Resumen **4**

1 Introducción **6**
1.1 Contexto y motivación . . . 6
1.2 Partes interesadas . . . 7
1.3 Objetivos . . . 7
1.4 Organización del informe . . . 7

2 Marco teórico **8**
2.1 Analizador Vectorial de Redes . . . 8
2.2 Parámetros de dispersión (S) . . . 8
2.2.1 Parámetro S11 . . . 9
2.2.2 Calibración S11 . . . 9
2.2.2.1 Errores . . . 9
2.2.2.2 Método SOL . . . 9
2.3 Acoplador Direccional . . . 10
2.3.1 Puente resistivo . . . 11
2.4 Buenas prácticas de diseño en RF . . . 12
2.5 Heterodinación de señales . . . 12
2.6 Procesamiento digital de señales . . . 13
2.6.1 Filtros digitales . . . 13
2.6.1.1 Diseño de filtros FIR mediante el método del enventanado . . . 14
2.6.2 Transformada Rápida de Fourier . . . 15

3 Proyecto **16**
3.1 Descripción general del producto . . . 16
3.2 Hardware . . . 16
3.2.1 Generadores de RF . . . 17
3.2.1.1 Filtro externo de lazo de control . . . 18
3.2.2 Oscilador . . . 19
3.2.3 Mezclador . . . 19
3.2.4 Acoplador direccional . . . 19
3.2.5 Amplificadores diferenciales . . . 19
3.2.6 Alimentación y conectores . . . 20
3.2.7 Implementación en PCB . . . 20
3.3 Firmware . . . 20
3.3.1 Cálculo de registros . . . 20
3.3.2 Transmisión SPI y secuencia de sincronización . . . 21

3.4 Software . . . 21
3.4.1 Flujo de barrido . . . 22
3.4.2 Estimación del S11 . . . 23
3.4.3 Calibración . . . 23
3.5 Interfaz de usuario . . . 24

4 Resultados **26**

5 Conclusiones y trabajos futuros **27**

6 Esquemáticos y código **28**
6.1 Esquemáticos Altium . . . 28
6.2 Firmware . . . 28
6.3 Software . . . 28
6.4 Interfaz de usuario . . . 28
6.4.1 Core . . . 28
6.4.2 Graphical User Interface . . . 35

Apéndices **48**

Bibliografía **50**

Siglas y abreviaturas

Nombre	Descripción
VNA	Analizador Vectorial de Redes
UHF	Ultra High Frequency
LAC	Laboratorio de Comunicaciones
ICYTE	Instituto de Investigaciones Científicas y Tecnológicas en Electrónica
FI	Frecuencia Intermedia
DUT	Device Under Test
RF	Radiofrecuencia
PCB	Printed Circuit Board
OL	Oscilador Local
FIR	Finite Impulse Response
FFT	Fast Fourier Transform
IC	Integrated Circuits
IA	Inteligencia Artificial

Resumen

El proyecto propone el diseño, implementación, calibración y ensayo de un Analizador Vectorial de Redes (VNA) de un puerto, que opere en una banda de frecuencias UHF (Ultra High Frequency) que va desde los 1,8 hasta los 2,2 GHz, adaptado para operar en conjunto con un sensor de glucemia no invasivo.

El principio de medición se basa en la existencia de una correlación entre la concentración de glucosa en sangre y sus propiedades eléctricas (permitividad relativa y conductividad) [1], por lo que es razonable suponer que las variaciones en la composición dieléctrica de una muestra de tejido biológico pueden alterar la distribución de los campos electromagnéticos presentes en un sensor de microondas.

La implementación se realiza en una placa de circuito impreso, con circuitos integrados especializados y aplicando técnicas de reflectometría, diseño en radiofrecuencia y manejo de emisiones electromagnéticas.

Se obtienen señales analógicas en la banda de audio para su adquisición en la placa de sonido de una PC. Luego, el análisis de las señales es puramente digital. Se aplica un procesamiento basado en la Transformada Rápida de Fourier (FFT) para obtener el coeficiente de reflexión del sistema bajo prueba en función de la frecuencia.

Capítulo 1

Introducción

1.1 Contexto y motivación

La diabetes es una afección crónica que se desencadena cuando el organismo pierde su capacidad de producir suficiente insulina o de utilizarla con eficacia para mantener los niveles de glucosa en la sangre dentro de los márgenes normales. La cantidad de adultos que la padecen se ha casi cuadruplicado entre 1980 y 2016, con un aumento en la prevalencia más acelerado en países de ingresos bajos y medios que en países de ingresos altos [2]. Para 2021 fue la causa directa de 1,6 millones de muertes [3]. Existen numerosos dispositivos que permiten realizar un control de la glucosa en sangre pero requieren, en general, de una punción en alguna parte del cuerpo para extraer una muestra de sangre. El mecanismo tradicional requiere depositar la muestra en una tira reactiva dentro de un medidor para obtener el valor de glucemia. Se recomienda, en promedio, que los pacientes realicen 4 mediciones al día (dependiendo de la estabilidad de los niveles o del régimen de tratamiento). Dado que la extracción suele ser dolorosa, además de que tiene un costo asociado al gasto de cada tira reactiva, la mayoría de los pacientes no logran tener un buen monitoreo de su glucemia. Por este motivo se ha incursionado en la glucometría semi-invasiva como los sistemas CGM (Continous Glucose Monitoring) [4] que obtienen mediciones del líquido intersticial y requieren la colocación de un sensor debajo de la piel que se debe reemplazar periódicamente, por lo que siguen siendo invasivos, incómodos de usar y descartables. El estudio de propuestas para la medición no invasiva ha avanzado en los últimos años, pero aún no existe una técnica ampliamente adoptada ni dispositivos comerciales fiables. Se ha observado que existe una correlación entre la concentración de glucosa en sangre y sus propiedades eléctricas (permitividad relativa y conductividad) [1], por lo que es razonable suponer que las variaciones en la composición dieléctrica de una muestra de tejido biológico pueden alterar la distribución de los campos electromagnéticos presentes en un sensor de microondas.

Por esta razón el Laboratorio de Comunicaciones (LAC), perteneciente al Instituto de Investigaciones Científicas y Tecnológicas en Electrónica (ICYTE), que depende de la UNMDP y del Consejo Nacional de Investigaciones Científicas y Técnicas (CONICET) tiene una línea de investigación cuyo objetivo es diseñar e implementar un prototipo que permita realizar una medición de glucemia, basado en un VNA y un sensor de microondas.

Un Analizador Vectorial de Redes es un instrumento de medición que permite caracterizar sistemas en alta frecuencia. La forma en que lo hace es haciendo incidir una señal de microondas en los puertos del sistema, y midiendo las señales que salen del resto de los puertos.

El Laboratorio busca optimizar, compactar y mejorar el rendimiento del VNA actual, por lo que requiere el diseño y desarrollo de un VNA de un puerto que esté adaptado a un sensor de microondas que resuena alrededor de los 2 GHz.

1.2 Partes interesadas

El Grupo de Investigación del LAC es quien solicita el desarrollo. Por esta razón, el equipo se utilizará bajo su supervisión, y sus posibles usuarios son:

- Ingenieros, técnicos y/o radioaficionados;
- Estudiantes de ingeniería.

1.3 Objetivos

El objetivo principal del proyecto es el diseño, desarrollo, calibración y ensayo de un VNA de bajo costo y tamaño compacto, que opere en la banda de frecuencias de UHF que abarca desde 1,8 a 2,2 GHz, orientado a la medición no invasiva de glucemia.

Los objetivos particulares son:

- Implementar en una placa de circuito impreso el VNA;
- Generar un barrido en la banda de frecuencias de interés;
- Implementar un sistema heterodino que permita trabajar con señales de frecuencia intermedia (FI) en la banda de audio;
- Digitalizar las señales obtenidas para aplicar un tratamiento digital que permita calcular el coeficiente de reflexión del sistema bajo prueba (DUT - *Device Under Test*);
- Implementar una rutina de calibración para eliminar errores sistemáticos del sistema;

1.4 Organización del informe

El informe se organiza de la siguiente manera: en el capítulo primero se introduce el proyecto y se brinda un punteo de los objetivos más importantes. El capítulo 2 se centra en el marco teórico, donde se explican las bases para entender el funcionamiento del dispositivo. El capítulo 3 describe el proyecto con su implementación. En el capítulo 4 se muestran los resultados y mediciones. El capítulo 5 se dedica a conclusiones y trabajos futuros, y el 6 exhibe los esquemáticos y códigos utilizados. Por último se anexan los apéndices en donde se encuentran: el estudio preliminar, el plan de trabajo, las especificaciones de requerimientos, funcional y técnica y el plan de pruebas.

Capítulo 2

Marco teórico

2.1 Analizador Vectorial de Redes

El Analizador Vectorial de Redes fue definido en sus inicios como un instrumento que permite medir los parámetros de redes eléctricas [5]. En su esquema más básico posee una fuente de radiofrecuencias (RF), que genera una señal de estímulo para el dispositivo bajo prueba, y un banco de receptores que miden las señales que salen del DUT y las comparan con la onda incidente. Los resultados se procesan y muestran al usuario por medio de alguna interfaz.

Dependiendo de los dispositivos que se buscan caracterizar, se determina la cantidad de puertos que debe tener el VNA. Un puerto permite medir el parámetro de reflexión. Con dos puertos, se puede medir además el de transmisión. Si posee un solo canal, se puede medir la transmisión inversa y la reflexión en la salida invirtiendo físicamente el dispositivo bajo prueba. Este paso se puede evitar agregando un canal, lo que incrementa el costo del VNA.

2.2 Parámetros de dispersión (S)

Los parámetros de dispersión, también llamados S por su nombre en inglés (*Scattering*) permiten caracterizar redes de dos o más puertos en función de sus ondas entrantes y salientes. Se especifican para una impedancia de referencia y a una frecuencia específica, y son útiles en altas frecuencias porque otros tipos de parámetros (como los Z o los Y) no pueden ser medidos directamente.

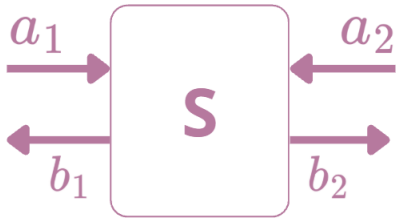


Figura 2.1: Parámetros S en una red de dos puertos

Según las ondas incidentes a_1 y a_2 y las reflejadas b_1 y b_2, los parámetros S se definen como:

$$S_{11} = \frac{b_1}{a_1}\Big|_{a_2=0};\ S_{12} = \frac{b_1}{a_2}\Big|_{a_1=0};\ S_{21} = \frac{b_2}{a_1}\Big|_{a_2=0};\ S_{22} = \frac{b_2}{a_2}\Big|_{a_1=0}$$

Para lograr que las a sean iguales a 0, el puerto al que pertenecen debe estar adaptado, garantizando que no haya ninguna señal incidente.

2.2.1 Parámetro S11

El parámetro S_{11} se conoce como el coeficiente de reflexión. Se puede definir en función de las ondas incidente y reflejada, o en función de la impedancia característica del sistema Z_0 y la impedancia de carga (es decir, del sistema bajo prueba):

$$S_{11}(DUT) = \Gamma = \frac{Z_L - Z_0}{Z_L + Z_0} \tag{2.2.1}$$

Expresado en dB, se expresa:

$$S_{11}(DUT)[dB] = 20.log(|\Gamma|) \tag{2.2.2}$$

2.2.2 Calibración S11

2.2.2.1 Errores

Los errores que pueden aparecer en una medición se pueden dividir en dos grandes clases: los errores sistemáticos y los aleatorios. Los primeros se pueden corregir, mientras que los segundos se encuentran en toda medición y están fuera del control del usuario.

En el caso de dispositivos electrónicos algunos errores sitemáticos son los producidos por variaciones en la potencia de salida de los generadores, pérdidas de inserción en materiales conductores y conectores, desfasajes ocasionados por diferencias en longitudes eléctricos o retardos en los circuitos integrados, reflexiones generadas por desadaptación de impedancias y deriva térmica. Algunos errores aleatorios pueden ser ruido térmico, de cuantización e interferencia electromagnética.

La calibración en un VNA permite corregir los errores sistemáticos, excluyendo: comportamientos alineales; repetitividad en conectores; errores en las cargas de calibración; y deriva térmica. Este último se puede corregir solo si la calibración se realiza inmediatamente antes de la medición.

2.2.2.2 Método SOL

En VNAs de un puerto se pueden simplificar los errores a un modelo de tres términos que describen la directividad, adaptación del puerto y errores de seguimiento (variaciones de amplitud y fase en función de la frecuencia)[6]. Usando tres cargas distintas de valores conocidos, se puede formar un sistema de tres ecuaciones con tres incógnitas y resolver para obtener los términos del error:

$$e_{00} + \Gamma_1\Gamma_{\mathrm{M}1}e_{11} - \Gamma_1\Delta_e = \Gamma_{\mathrm{M}1} \tag{2.2.3a}$$
$$e_{00} + \Gamma_2\Gamma_{\mathrm{M}2}e_{11} - \Gamma_2\Delta_e = \Gamma_{\mathrm{M}2} \tag{2.2.3b}$$
$$e_{00} + \Gamma_3\Gamma_{\mathrm{M}3}e_{11} - \Gamma_3\Delta_e = \Gamma_{\mathrm{M}3} \tag{2.2.3c}$$

Donde Γ_{M} es el parámetro de reflexión medido para cada carga, e_{00} es la directividad, e_{11} la adaptación del puerto, $e_{10}e_{01}$ el error de seguimiento (*tracking*) y $\Delta_e = e_{00}e_{11} - (e_{10}e_{01})$. Los términos del error se pueden observar en la figura 2.2.

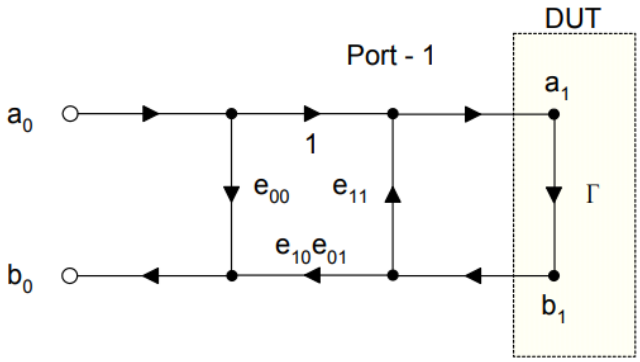


Figura 2.2: Términos de error para un VNA de un puerto

Si las cargas independientes son un circuito abierto, cortocircuito y carga de 50Ω entonces el método de calibración se denomina SOL (Short, Open, Load) y las ecuaciones (2.2.3) se simplifican a:

$$e_{00} - \Gamma_{\mathrm{MS}} e_{11} + \Delta_e = \Gamma_{\mathrm{MS}} \tag{2.2.4a}$$
$$e_{00} + \Gamma_{\mathrm{MO}} e_{11} - \Delta_e = \Gamma_{\mathrm{MO}} \tag{2.2.4b}$$
$$e_{00} = \Gamma_{\mathrm{ML}} \tag{2.2.4c}$$

Luego el Γ real se calcula como:

$$\Gamma = \frac{\Gamma_M - e_{00}}{\Gamma_M e_{11} - \Delta_e} \tag{2.2.5}$$

2.3 Acoplador Direccional

En la mayoría de diseños en electrónica, el acoplamiento electromagnético entre líneas de transmisión se considera indeseable porque interfiere con la señal que se busca transmitir. Una excepción es el caso de acopladores, donde el acoplamiento es un parámetro de diseño que permite medir la señal de una línea con un alto aislamiento, es decir, sin interrumpir su paso. Además se pueden diseñar un par de lineas acopladas que verifiquen que en uno de sus puertos la señal incidente se anula, generando un punto que permite medir la onda reflejada.

Las principales propiedades que caracterizan un acoplador son la directividad, el aislamiento, el acoplamiento y la pérdida de inserción [7]. La directividad mide la capacidad de separar las ondas que se propagan en distintas direcciones. El aislamiento, la potencia que llega al puerto desacoplado. El acoplamiento indica la proporción de potencia que se entrega al puerto acoplado. La pérdida de inserción muestra cuánta potencia (que se debería transmitir) se pierde por ser entregada a los puertos acoplados y aislados, y las pérdidas en los materiales (dieléctrico y resistividad de las pistas).

Un acoplador ideal es un dispositivo pasivo, que posee directividad y aislamiento infinitos.

Existen acopladores direccionales como el HP 778d [8] que posee una directividad mayor a 36dB y pérdidas por inserción menores a 0,6dB, pero tiene un ancho de banda que permite hacer mediciones entre 100 MHz y 2 GHz, y sus dimensiones físicas y peso hacen que su utilidad se limite a mediciones en laboratorio. Para soluciones integradas hay alternativas comerciales como el ZABDC20-322 de Minicircuits [9] pero su costo hace que el diseño no sea escalable.

2.3.1 Puente resistivo

Una solución compacta, económica y con un ancho de banda teórico infinito es el puente resistivo tipo Wheatstone. Como se puede observar en la figura 2.3 es una red simple de comparación de impedancias, formada por cuatro resistencias sobre las cuales se inyecta una señal de referencia en la parte superior. Una de las resistencias es variable (Zx), mientras que el resto son iguales entre sí. Se utiliza un voltímetro para medir la diferencia de potencial entre las ramas. Si la resistencia variable es de 50Ω, el puente queda balanceado y el voltímetro indica un cero. Si es menor a 50Ω, la rama izquierda presenta un voltaje menor a la derecha, mientras que para valores mayores ocurre lo opuesto. La tensión en el nodo de referencia V_A resulta:

$$V_A = V_{inc} \cdot \frac{Z_0}{Z_0 + Z_0} = \frac{V_{inc}}{2} \tag{2.3.1}$$

Mientras que la tensión en el nodo de medición V_B es:

$$V_B = V_{inc} \cdot \frac{Z_x}{Z_0 + Z_x} \tag{2.3.2}$$

Por lo tanto, la señal desbalanceada o tensión diferencial reflejada $v_{diff}(t)$ entre ambos nodos se define como:

$$v_{diff}(t) = v_B(t) - v_A(t) v_{diff}(t) = v_{inc}(t) \left[\frac{Z_x}{Z_0 + Z_x} - \frac{1}{2} \right] \tag{2.3.3a}$$

Reescribiendo la resta interna con común denominador $\frac{Z_x}{Z_x+Z_0} - \frac{1}{2} = \frac{1}{2}\left[\frac{Z_x - Z_0}{Z_x+Z_0}\right]$ entonces:

$$v_{diff}(t) = \frac{v_{inc}(t)}{2} \underbrace{\left[\frac{Z_x - Z_0}{Z_x + Z_0} \right]}_{\text{Coeficiente de reflexión } (\Gamma)} = \frac{v_{inc}(t)}{2} \cdot \Gamma \tag{2.3.4}$$

Si la señal incidente es sinusoidal es posible además caracterizar cargas reactivas midiendo el desfasaje de la señal Zx respecto de la incidente. Este es el principio que permite al puente comportarse como un acoplador direccional: la diferencia de potencial entre las ramas es función de la onda reflejada equivalente.

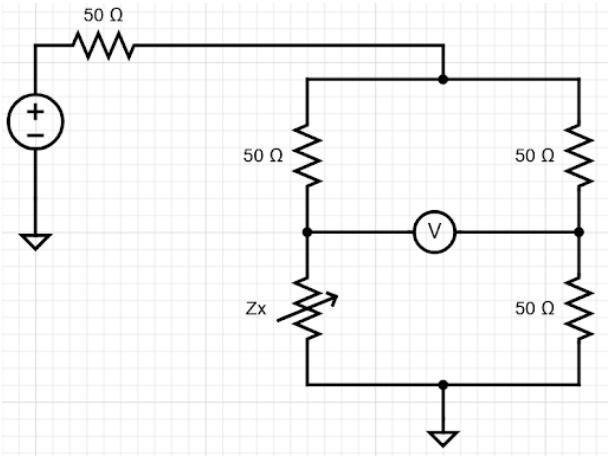


Figura 2.3: Esquema básico de un puente de Wheatstone

Para una aplicación de reflectometría, la magnitud de la impedancia de entrada al DUT haría las veces del Zx a caracterizar en este esquema simplificado. Esto, además de la capacidad de lograr directividad y ancho de banda elevados, lo vuelve una alternativa muy conveniente a la hora de implementar un VNA que funcione con buena sensibilidad en un rango amplio de frecuencias.

2.4 Buenas prácticas de diseño en RF

Al momento de rutear un PBC (*Printed Circuit Board* - Placa de Circuito Impreso) se deben tener en consideración algunas buenas prácticas que ayudan a mantener la integridad de las señales, la impedancia de las líneas y evitar la interferencia electromagnética, sobre todo cuando se trabaja en RF.

Se debe garantizar que las pistas cumplan con la impedancia requerida de 50Ω. Esto implica calcular el ancho de la línea usando las ecuaciones de diseño según el tipo de línea y las características de los materiales a utilizar (generalmente brindadas por el fabricante de PCB). Se pueden encontrar calculadoras online o integradas en los software de diseño con este fin. Se deben evitar ángulos rectos, que causan discontinuidades y aumentan la capacitancia parásita.

El plano de masa que se encuentra debajo de las pistas de RF debe ser continuo, para que la corriente de retorno pueda seguir el camino de menor inductancia. Por esta razón no se recomienda rutear en el plano inferior al de las rutas de RF, pero si se necesita hacerlo no se deben atravesar las regiones donde en la capa superior haya trazos de RF.

Se deben usar las técnicas *vía stitching* y *vía shielding*. La segunda se usa para reducir la interferencia electromagnética y el crosstalk [10], mientras que la primera "ata" los planos de masa de distintas capas, ayudando a mantener baja impedancia y caminos de retorno cortos. Además evita resonancias dentro de la placa.

Los capacitores de desacople se deben ubicar lo más cerca posible de los integrados, de forma de poder suministrar rápidamente los picos de corriente que demandan los integrados que la fuente de alimentación no puede entregar [11]. También deben utilizarse cuando se suministra alimentación a un trazo a través de vías.

Los trazos deben ser lo más cortos posible, para minimizar la atenuación de las señales y reflexiones indeseadas.

2.5 Heterodinación de señales

Las señales que se intentan medir en un VNA son de naturaleza analógica, por lo que para realizar el cálculo del parámetro de reflexión o S11 primero se debe poder medir su amplitud y fase. Existen integrados que pueden obtener la magnitud de una onda de RF como el HMC601 [12] pero se pierde información de la fase. Por eso se busca una forma de bajar la frecuencia de la señal para poder adquirirla de forma digital.

Un esquema heterodino se basa en la combinación de una señal de RF de interés con otra señal generada localmente por un oscilador local (OL o LO por sus siglas en inglés) desplazada en frecuencia un valor constante, llamado frecuencia intermedia (FI).

Cuando se multiplican idealmente en el dominio del tiempo dos señales sinusoidales $v_{RF}(t) = A_{RF}\cos(2\pi f_{RF}t)$ y $v_{OL}(t) = A_{LO}\cos(2\pi(f_{RF}+f_{IF})t)$ (suponiendo que tienen la misma fase inicial), a la salida existen componentes espectrales en la suma y diferencia de las frecuencias originales:

$$v_{mix}(t) = k \cdot v_{RF}(t) \cdot v_{LO}(t) \tag{2.5.1a}$$
$$v_{mix}(t) = kA_{RF}A_{LO}\cos(2\pi f_{RF}t)\cos(2\pi(f_{RF}+f_{IF})t) \tag{2.5.1b}$$
$$\tag{2.5.1c}$$

Reescribiendo con la identidad trigonométrica $\cos(\alpha)\cos(\beta) = \frac{1}{2}\left[\cos(\alpha-\beta)+\cos(\alpha+\beta)\right]$ entonces:

$$v_{mix}(t) = \frac{kA_{RF}A_{LO}}{2}\left[\underbrace{\cos\left(2\pi f_{FI}t\right)}_{\text{Componente Resta (FI)}} + \underbrace{\cos\left(2\pi(2f_{RF}+f_{FI})t\right)}_{\text{Componente Suma (Alta Frecuencia)}}\right] \tag{2.5.2a}$$

Mediante un filtrado se puede aislar la componente diferencia, logrando así trasladar la información de la señal de RF hacia la FI:

$$v_{\mathrm{IF}}(t) = \frac{kA_{\mathrm{RF}}A_{LO}}{2}\cos\left(2\pi f_{\mathrm{IF}}t\right) \tag{2.5.3}$$

Utilizando un mezclador doble se pueden heterodinar dos señales con un solo oscilador local, como muestra la figura 2.4 . De esta forma obtenienen las señales incidente y reflejada, conservando su amplitud y fase, en una frecuencia de audio que facilite su adquisición y procesamiento.

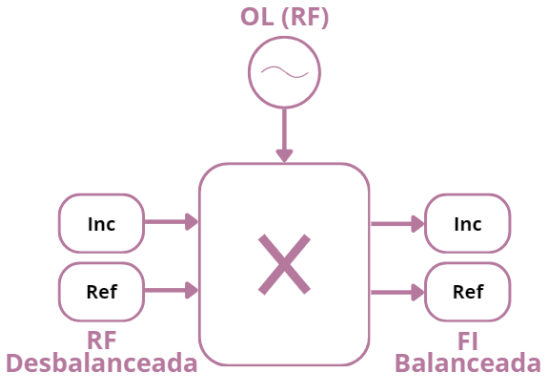


Figura 2.4: Heterodinación de señales

2.6 Procesamiento digital de señales

Para poder aplicar un procesamiento digital cualquier señal analógica continua en el tiempo debe ser muestreada, es decir convertirla en una secuencia de números de precisión finita. El proceso implica una pérdida de información de los intervalos intermedios, pero según el teorema de Nyquist-Shannon una señal de banda limitada puede reconstruirse inequívocamente a partir de sus muestras si la frecuencia de muestreo f_s cumple la condición $f_s \geq 2f_{\max}$, donde $f_{\max}$ representa la componente frecuencial de mayor valor de la señal [13].

Una vez que la señal está digitalizada, para poder extraer su amplitud y fase, sabiendo que se encuentra a una frecuencia de valor FI, se debe aplicar un filtrado digital y realizar la Transformada Rápida de Fourier (FFT - *Fast Fourier Transform*)

2.6.1 Filtros digitales

El objetivo principal de la etapa de filtrado es aislar la señal de FI eliminando el ruido de banda ancha, las interferencias de baja frecuencia y armónicos de orden superior.

La señal muestreada $x[n]$ pasa por un filtro pasabanda digital caracterizado por una respuesta al impulso $h[n]$. La operación se define mediante la convolución discreta:

$$y[n] = x[n] * h[n] = \sum_{k=0}^{M} h[k]\, x[n-k] \tag{2.6.1}$$

donde M representa el orden del filtro. Los coeficientes $h[k]$ se diseñan para configurar una banda de paso en torno a la frecuencia de FI.

Los filtros digitales se pueden clasificar en dos grandes familias según la naturaleza de su respuesta al impulso: filtros de respuesta finita (*Finite Impulse Response*, FIR) y filtros de respuesta infinita (*Infinite Impulse Response*, IIR).

Un filtro IIR se caracteriza por una respuesta al impulso de duración infinita, que se logra mediante una estructura recursiva con realimentación de las salidas anteriores. Su ecuación en diferencias se expresa como:

$$y[n] = \sum_{k=0}^{M} b_k \, x[n-k] + \sum_{k=1}^{N} a_k \, y[n-k] \tag{2.6.2}$$

La presencia de los coeficientes a_k introduce polos en el plano z fuera del origen. Si bien permiten lograr bandas de transición abruptas con órdenes bajos, presentan dos desventajas fundamentales: no garantizan fase lineal en toda la banda de paso y pueden ser inestables si sus polos migran fuera del circulo unitario debido a cuantización de precisión finita.

Por otro lado, un filtro FIR presenta una respuesta al impulso de duración finita N_{taps} (comúnmente denominada número de coeficientes o *taps*), siendo $h[n] = 0$ para $n < 0$ y $n \geq N_{\text{taps}}$. Su ecuación en diferencias es puramente no recursiva:

$$y[n] = \sum_{k=0}^{M} b_k \, x[n-k] = \sum_{k=0}^{N_{\text{taps}}-1} h[k] \, x[n-k] \tag{2.6.3}$$

donde $M = N_{\text{taps}} - 1$ representa el orden del filtro. Su transferencia $H(z)$ es causal, estable y realizable. Todos sus polos se ubican de manera trivial en el origen ($z = 0$), lo que garantiza estabilidad incondicional.

Si la secuencia de coeficientes $h[n]$ cumple la condición de simetría par alrededor de su centro ($h[n] = h[N_{\text{taps}} - 1 - n]$), la respuesta en frecuencia presenta la forma:

$$H(e^{j\omega}) = |H(e^{j\omega})| \, e^{-j\omega\tau_g} \tag{2.6.4}$$

donde el retardo de grupo τ_g es estrictamente constante para todas las frecuencias:

$$\tau_g = \frac{N_{\text{taps}} - 1}{2} \quad [\text{muestras}] \tag{2.6.5}$$

Un retardo de grupo constante asegura que el filtro no introduce distorsión de fase no lineal, preservando la diferencia de fase relativa entre el canal de referencia (V_{ref}) y el de medición (V_{med}).

2.6.1.1 Diseño de filtros FIR mediante el método del enventanado

Las forma más simple de diseño de filtros FIR es el llamado método del enventanado (*window design method*) [14]. Este método consiste en truncar y desplazar la respuesta al impulso ideal de duración infinita, $h_d[n]$ (correspondiente a un filtro pasabanda ideal centrado en f_{FI}), multiplicándola punto a punto por una función ventana finita $w[n]$ de longitud N_{taps}:

$$h[n] = h_d[n] \cdot w[n], \quad 0 \leq n \leq N_{\text{taps}} - 1 \tag{2.6.6}$$

Una de las posibles ventanas a utilizar es la Hamming, definida como:

$$w[n] = 0{,}54 - 0{,}46 \cos\left(\frac{2\pi n}{N_{\text{taps}} - 1}\right), \quad 0 \leq n \leq N_{\text{taps}} - 1 \tag{2.6.7}$$

La multiplicación en el dominio del tiempo equivale a la convolución periódica de la respuesta en frecuencia ideal con el espectro de la ventana, $W(e^{j\omega})$. La ventana de Hamming tiene como características principales:

- Atenuación en banda de rechazo: Proporciona un nivel de lóbulo secundario pico de aproximadamente -41 dB y una atenuación mínima en la banda de rechazo de -53 dB.
- Suavizado del fenómeno de Gibbs: Elimina las oscilaciones de sobrepico en los bordes de la banda de paso típicas del truncamiento rectangular estricto, a cambio de un lóbulo central más ancho.

2.6.2 Transformada Rápida de Fourier

Para determinar la amplitud y fase de las señales de FI, las secuencias filtradas en el dominio del tiempo se transforman al dominio de la frecuencia mediante la Transformada Discreta de Fourier (DFT), calculada a través del algoritmo de la Transformada Rápida de Fourier (FFT). Para una secuencia de N muestras, la expresión de análisis viene dada por [13]:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, w[n]\, e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \ldots, N-1 \tag{2.6.8}$$

donde $w[n]$ representa la función de la ventana aplicada a la secuencia temporal.

Dado que la captura se realiza sobre una ventana de observación finita, si la duración del intervalo de adquisición no contiene un número entero exacto de ciclos de la señal, se produce el fenómeno de fuga espectral (*spectral leakage*). Este fenómeno dispersa la energía de la frecuencia fundamental hacia los bins frecuenciales adyacentes, degradando la precisión en la estimación de amplitud y fase. Para mitigar este efecto, se aplica una función de enventanado $w[n]$. La ventana *Flattop* se caracteriza por presentar un lóbulo principal ancho y plano; esta propiedad minimiza el error de estimación de amplitud, mejorando la exactitud en la medición de niveles de tensión.

Capítulo 3

Proyecto

3.1 Descripción general del producto

Como describe la sección 2 de la Especificación de Requerimientos (**??**) el sistema debe generar un barrido de frecuencias en el rango de 1,8 a 2,2 GHz. La señal se debe inyectar en el DUT y se debe medir la onda reflejada. Ambas ondas (incidente y reflejada) deben bajarse a una frecuencia intermedia para luego adquirirlas y digitalizarlas con el fin de realizar el procesamiento correspondiente. Un diagrama en bloques detallado del sistema se puede observar en la figura 3.1. Los datos a asumir, requerimientos y pautas de diseño se pueden encontrar en la Especificación de Requerimientos (**??**).

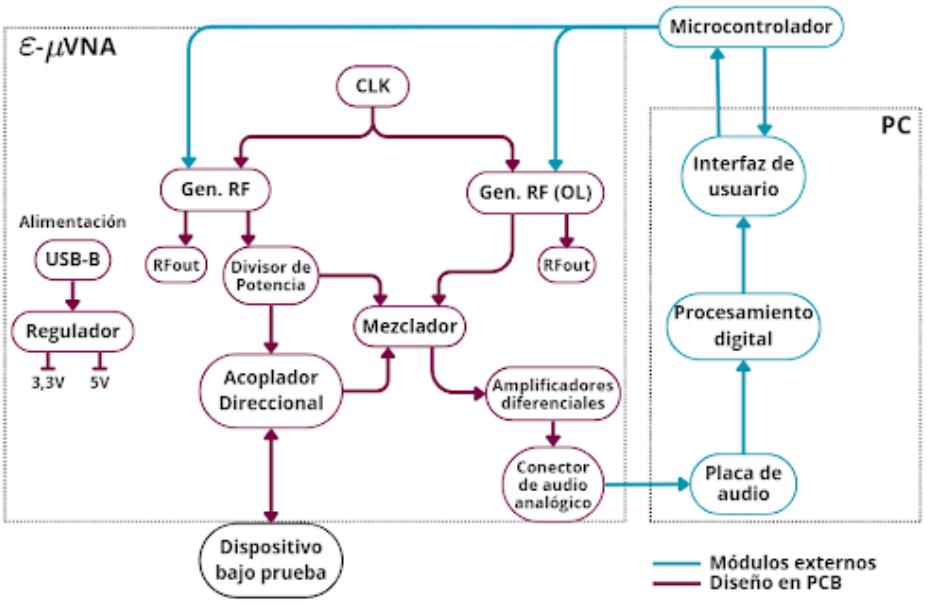


Figura 3.1: Diagrama en bloques del sistema

A continuación, se detalla la solución del hardware, software, firmware e interfaz de usuario.

3.2 Hardware

El hardware se diseñó en su totalidad en el software Altium Designer, con una licencia estudiantil gracias al programa Altium Student Lab [15]. Se creó una librería con los esquemáticos, footprints y modelos 3D de los componentes a utilizar. Se dividió el proyecto en esquemáticos que representan los distintos bloques funcionales, que se pueden observar en el capítulo "Es-

quemáticos y códigos" (6.1). Se puede encontrar una descripción general de cada bloque en la Especificación Técnica (**??**).

3.2.1 Generadores de RF

Como explica la sección 2.1.2 de la Especificación Técnica (**??**) se sintetizan con los circuitos integrados (IC - *Integrated Circuits*) ADF4351, un sintetizador de banda ancha con VCO integrado, con frecuencia de salida configurable de 40 a 4400 MHz y potencia de salida máxima de 5dBm [16]. El hardware posee dos; uno incide directamente en el acoplador direccional, desviando una parte de su potencia hacia el mezclador para medir la señal incidente. El esquemático de su implementación se puede ver en **??**. El otro se utiliza como oscilador local para el esquema heterodino (esquemático en **??**).

El diagrama en bloques del chip se puede observar en la figura 3.2:

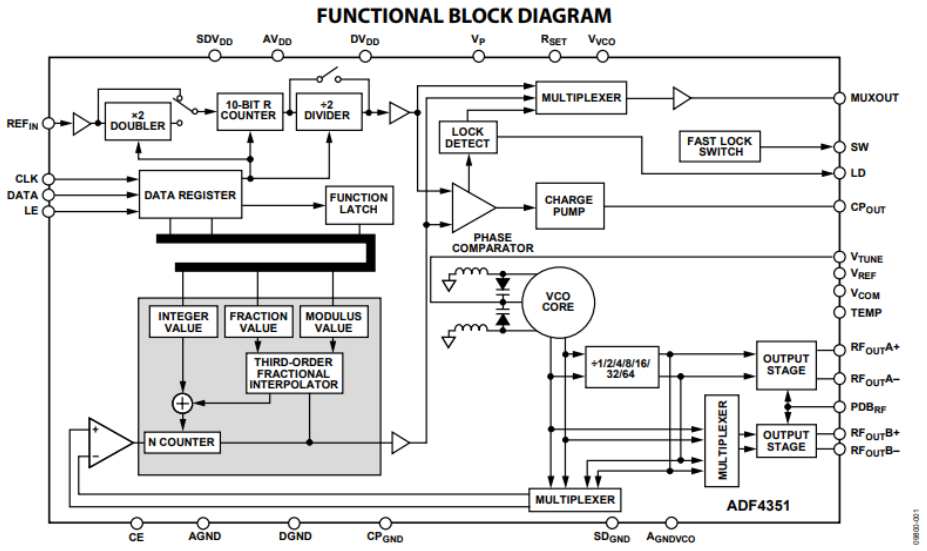


Figura 3.2: Diagrama en bloques del integrado ADF4351

Algunas consideraciones que se tuvieron en cuenta al momento de diseñar y rutear fueron:

- Tensión de alimentación (Vcc) de 3,3 V, ya que soporta de 3 a 3,6 V. Esta tensión debe ir a pines que se encuentran distribuidos en todos los lados del integrado, por lo que al estar ingresando con señales de RF se debe tener especial cuidado en el ruteo para mantener la integridad de las señales.

- El pad central expuesto del IC debe ser soldado a un pad GND (tierra) de al menos su mismo tamaño por razones térmicas, por indicación del datasheet [16]. Además, se sugiere colocar vías para mejorar el comportamiento térmico.

- La salida de RF es de colector abierto, por lo que para funcionar necesitan una etapa de pull-up: la forma más sencilla (y de banda ancha) de hacerlo es con una resistencia de 50 Ω a V_{VCO}. De esta forma la salida del circuito es una potencia que equivale al valor registrado en el registro 4 (R4) (ver sección firmware 3.3). Por la misma razón, por usarse en modo desbalanceado, la salida de la rama que se deja sin usar debe terminarse en una resistencia de 50 Ω. En este caso, se coloca una línea de transmisión hacia un conector SMA para mediciones auxiliares que en caso de no utilizarse, debe adaptarse.

- Es importante que la señal V_{TUNE}, una continua que es la entrada de control del VCO, esté libre de ruido, ya que determina la frecuencia de salida y es vital para el correcto enganche.

- La hoja de datos recomienda poner capacitores de desacople lo más cerca posible de los pines V_P, AV_{DD}, V_{VCO}, $TEMP$, V_{COM}, V_{REF}, DV_{DD}, y SDV_{DD}.
- Los pines que se utilizan para programar el integrado son una entrada de alta impedancia CMOS, con un límite máximo de 3,6 V. Para protegerlas por incompatibilidad de niveles lógicos, se utilizan atenuadores en cada una de las entradas (CLK, CE, LE, DATA)**??**.
- Se coloca un LED en la salida de detección de enganche, para visualizar rápidamente si el integrado está enganchado.

3.2.1.1 Filtro externo de lazo de control

El filtro de lazo se diseña en la aplicación del fabricante, Analog Devices, ADISIMPLL 5.9 [17]. Se busca que cumpla con un margen de fase mínimo de 45° para preservar la estabilidad del sistema, y que el ancho de banda sea lo mayor posible para disminuir el tiempo de enganche, a costa de un incremento en el ruido que ingresa al lazo que proviene de la referencia, del detector de fase y la bomba de carga.

El esquema final se puede observar en el entorno del software en la figura 3.3.

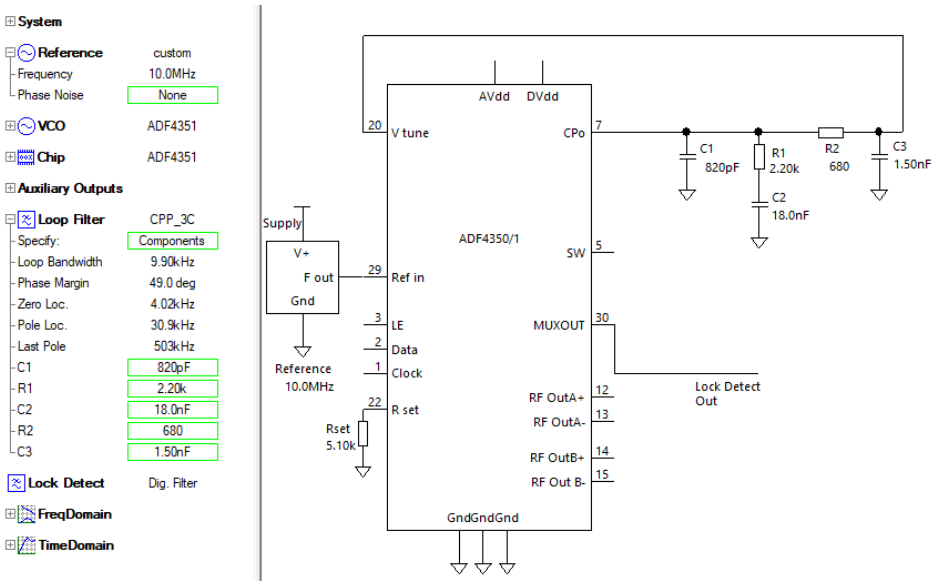


Figura 3.3: Diseño del filtro de lazo externo

La respuesta en frecuencia de todo el sistema (ADF4351 con el filtro diseñado), simulada para 2 GHz, se muestra en la figura 3.4

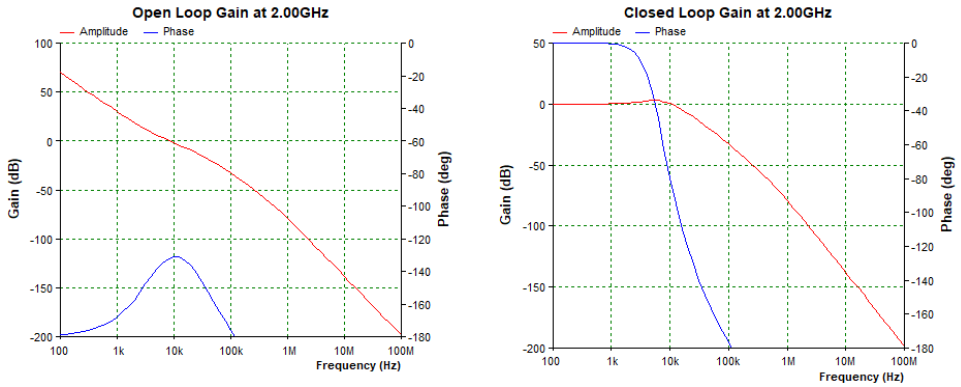


Figura 3.4: Respuesta en frecuencia de los generadores

3.2.2 Oscilador

La referencia para los generadores de RF se compone de un oscilador a cristal ECS-2200BX-100 [18] que proporciona una salida cuadrada y un buffer con salida open drain 74LVC2G07W6-7 [19]. Este permite no solo aumentar el fan-out (dividiendo la señal en dos caminos sin degradarla) sino realizar un reacondicionamiento. De esta manera se reduce el crosstalk (aislando las entradas de los generadores entre sí) y el ringing (oscilaciones de reflexión), garantizando flancos más limpios y rápidos. Se colocan dos resistencias de 47 Ω para absorber las ondas reflejadas existan en la línea debido a que la entrada de referencia de los ADF es de alta impedancia y limitar la corriente de conmutación (ayudando a reducir el ruido de alta frecuencia). **??**

3.2.3 Mezclador

El mezclador se implementa con el IC ADL5802, un mezclador activo de alta linealidad, de dos canales, doble balanceado, con buffers de OL integrados para proveer conversión de frecuencia con alto rango dinámico. Su esquemático se puede observar en **??**

Su función principal es realizar la heterodinación de las señales de radiofrecuencia. Se aplica la señal proveniente del DUT a la entrada de RF, y la del ADF4351 que actúa de OL a la entrada correspondiente.

El integrado se alimenta con una tensión continua de 5 V. Sus salidas en FI son de colector abierto con una impedancia diferencial de 200 Ω, por lo que se colocan resistencias de pull-up de 50 Ω conectadas a la línea de 5 V para su correcta polarización.

Las salidas diferenciales del integrado permiten obtener las componentes correspondientes a las señales incidente y reflejada, identificadas como IF_INC+, IF_INC-, IF_MED+ e IF_MED- en el esquemático. A la salida de cada canal se integran filtros pasabajo RC simples, calculados para atenuar la componente suma de alta frecuencia ($2f_{\mathrm{RF}} + f_{\mathrm{FI}}$) y dejar pasar únicamente las señales diferenciales de baja frecuencia correspondientes a la FI. Estas señales se destinan a las etapas posteriores de adquisición y procesamiento.

Se disponen capacitores de desacople en la alimentación y un divisor resistivo en la entrada VSET, que es el control de polarización para alto IP3 (lo que mejora la linealidad de entrada). Se aplica entre 4 y 5 V para mejorar la performance de IP3, y entre 2 y 3 V para mejorar la figura de ruido. Actualmente el valor está seteado en aproximadamente 4,1V.

3.2.4 Acoplador direccional

Basado en el puente de Wheatstone explicado en el marco teórico, permite dirigir la señal generada hacia el sensor, mientras que una parte de la señal reflejada por la carga se obtiene mediante el puerto acoplado. De esta manera, es posible acceder a ambas señales sin necesidad de interrumpir directamente el camino principal de transmisión.

El puente diseñado, como se muestra en el esquemático (**??**), agrega dos transformadores de RF TC1-1-13M+ [20] en cascada, que permiten medir la señal reflejada de manera desbalanceada y filtrar el ruido de modo común que provocaría usar solo uno. Las resistencias de 68Ω logran que la impedancia vista desde el puerto de salida (J1) sea 50Ω.

3.2.5 Amplificadores diferenciales

Al tener salidas diferenciales y de muy poca amplitud, se debe agregar una etapa de acondicionamiento que permita obtener una señal desbalanceada y apta para ser procesada por la placa de sonido de la PC. Esta etapa utiliza amplificadores operacionales MCP6023 de Microchip Technology [21]. Se configuran como restadores, permitiendo referir las señales a masa y proveer la ganancia necesaria para trabajar dentro del rango dinámico de la placa de sonido.

3.2.6 Alimentación y conectores

Para alimentar la placa se dispone de una selección (a través de un jumper) de pines para utilizar una fuente de laboratorio o un USB-B para conectarlo a alguna salida de PC o cargador de 5V. Se agrega un regulador de tensión UA78M33CDCYR [22] de Texas Instruments que provee los 3,3V necesarios para alimentar a los generadores de RF y oscilador.

Los conectores que posee son del tipo SMA para mediciones de RF, conexión con el DUT y entrada de referencia auxiliar; conector de audio analógico para llevar las señales a la placa de sonido de la PC por medio de la entrada line-in, y tiras de pines que por medio de cables puente se conectan con el Arduino para escribir los registros de los generadores de RF. Además se ubican pares de pines para medir la detección de enganche en cada ADF4351.

3.2.7 Implementación en PCB

En principio se planteó un modelo de 2 capas, que se puede observar en la figura **??**. Tiene FR4 como dieléctrico de 0,8 mm de espesor, con un espesor total de 0,9 mm, y un tamaño de 10 x 9,5 mm^2. Pero el ruteo del pin V_{TUNE} de los ADF4351 y la alimentación en la parte central de la placa, donde había un sector rodeado de pistas de RF, imponía el uso de varios puentes. Se implementaron con postes y cables de arduino, pero no es la manera óptima porque son sensibles a interferencias. Por este motivo, se decidió hacer un modelo de 4 capas, que permitiera tener un plano de masa continuo bajo las pistas de RF y poder rutear usando la capa inferior, además de tener una tercera capa para el manejo de la alimentación. A este modelo se lo llamó $\epsilon-\mu VNA$ v2, tiene un tamaño de 8,7 x 7,8 mm^2 y se lo puede observar en la figura **??**.

3.3 Firmware

El firmware se programó en lenguaje C++ en el entorno Arduino IDE. Se puede leer en la sección "códigos" (6.2). Está desarrollado para un microcontrolador Arduino UNO y se encarga del control, configuración dinámica y sincronización de los sintetizadores ADF4351.

El microcontrolador actúa como un esclavo de ejecución que recibe las órdenes de barrido desde la PC a través de una interfaz serie. Opera a una velocidad de 115200 baudios. Al ser inicializado en la función de arranque (*setup*), configura los pines de control, inicializa el bus SPI y transmite la cadena "Listo" para confirmar que el sistema está disponible.

Al recibir una frecuencia f, el algoritmo configura el primer sintetizador (ADF1) exactamente en f (canal de RF) y el segundo sintetizador (ADF2) en $f+0.010$ MHz (canal LO), introduciendo de forma automática el desplazamiento constante para la FI. Al finalizar la reconfiguración de ambos integrados, responde a la PC con la cadena "OK".

3.3.1 Cálculo de registros

Para cada valor de frecuencia solicitado, la función "calcularRegistros" determina matemáticamente el contenido de los 6 registros de 32 bits (R0 a R5) de cada chip ADF4351 según las especificaciones del fabricante:

- Divisor de Salida de RF (RF Divider):Selecciona el divisor adecuado (1, 2, 4, 8 o 16) según el rango de frecuencia de trabajo (desde menos de 275 MHz hasta más de 2200 MHz) para garantizar que el VCO interno funcione en su rango de 2.2 GHz a 4.4 GHz.
- Frecuencia del Detector de Fase (f_{PFD}): A partir del oscilador de referencia externo de 10 MHz, aplica la división previa por 2 (REF_DIV2 = true) y el contador R (R_COUNTER = 10), fijando una frecuencia de comparación en el PFD de 500 kHz ($f_{\mathrm{PFD}} = 0.5$ MHz).
- Síntesis N-Fraccionario: Divide la frecuencia del VCO por f_{PFD} para calcular la parte entera (INT) y la parte fraccionaria (FRAC) considerando un módulo fijo MOD = 1000.

- Configuración de Registros Internos:
  - R0: Almacena los bits correspondientes a 'INT' y 'FRAC'.
  - R1: Configura el *prescaler* en 8/9 y fija el módulo ('MOD') y la fase ('PHASE').
  - R2: Establece la polaridad positiva del detector de fase, habilita los divisores de referencia y fija la corriente de la bomba de carga (*charge pump*) en 5.00 mA.
  - R3: Habilita el modo de enganche rápido (*fast lock*).
  - R4: Selecciona el divisor de RF, habilita la salida principal, fija el reloj de selección de banda (*Band Select Divider*) y configura la potencia de salida en +5 dBm.
  - R5: Selecciona la operación del pin de enganche (*Lock Detect*). El resto de los bits son un registro de control fijado según recomendación de la hoja de datos.

3.3.2 Transmisión SPI y secuencia de sincronización

El envío de los registros hacia los integrados se realiza mediante el bus SPI de hardware y líneas de control independientes. Utiliza pines dedicados para la habilitación Latch Enable (LE) y Chip Enable (CE). Cada registro se envía dividiendo los 32 bits en 4 bytes (MSB primero) sobre la interfaz SPI, generando un pulso positivo de 2 microsegundos en la línea 'LE' para almacenar los datos en el integrado. Los registros se escriben en orden inverso, desde R5 hasta R0, debido a que la transferencia del registro R0 actúa como el disparador (*trigger*) que actualiza efectivamente la frecuencia del VCO.

Para mejorar la coherencia y estabilidad de fase en el canal del Oscilador Local (ADF2), se aplica un procedimiento en dos etapas: se escriben primero los registros R5 a R1, se genera un retardo intencional de 10 milisegundos para permitir la estabilización parcial del lazo, y finalmente se escribe el registro R0 para cerrar la sintonía.

3.4 Software

Como se explica en la sección 4 de la Especificación Técnica **??** el software se programa en lenguaje Python en el entorno Visual Studio Code. El código se encuentra en la sección **??**. Un diagrama de bloques general se puede ver en la figura 3.5. Las funciones principales que cumple son:

- Gestionar el barrido en frecuencia;
- Controlar la adquisición de audio;
- Ejecutar el procesamiento digital de señales en tiempo real;
- Implementar el modelo de calibración SOL de un puerto para obtener el coeficiente de reflexión (Γ) del DUT;
- Mostrar resultados.

Las configuraciones iniciales son: Puerto COM, baudrate, frecuencias de inicio, fin y paso, offset (es decir, frecuencia de audio a utilizar), frecuencia de muestreo para la adquisición, duración de la captura y su descarte inicial debido al transitorio, los nombres de los archivos de salida, la frecuencia central del filtro FIR (que debe coincidir con la frecuencia de audio), su ancho de banda, numtaps y ventana a utilizar.

La función principal (*main*) genera el vector de frecuencias a partir de las frecuencias de inicio, fin y paso. Luego diseña el filtro pasabanda. Chequea en el directorio si hay algún archivo de calibración cuyo nombre, que está predefinido con las configuraciones de frecuencia coincida con las ingresadas por el usuario. En caso afirmativo, pregunta si se desea utilizar o se prefiere realizar una nueva calibración. Dependiendo de la respuesta del usuario por terminal, entra al bucle de calibración o carga los parámetros desde el archivo. Procede a realizar la conexión con el Arduino. Si se debe calibrar, realiza tres barridos para obtener los valores de Γ_{MED} y completar la matriz de calibración. Estos barridos corresponden a cortocircuito, circuito abierto y carga, en ese orden. Una vez que se hacen los barridos calcula los errores y se guardan en el archivo CSV (archivo de texto plano donde los valroes están separados por comas). Realiza el barrido del DUT y cierra el puerto serie. Aplica la corrección, calcula la fase en grados y el módulo en dB y se guardan en vectores separados. Crea un archivo CSV con estos datos y muestra los resultados en gráficos de módulo y fase en función de la frecuencia.

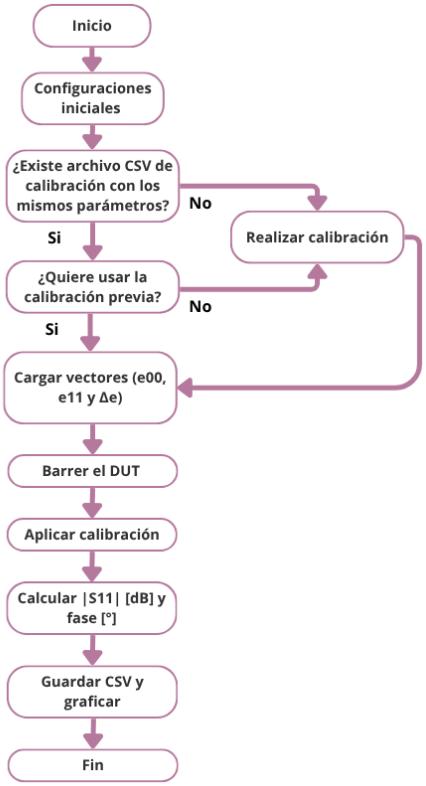


Figura 3.5: Diagrama de flujo general del software.

3.4.1 Flujo de barrido

La función "barrido" se encarga de enviar los datos de la frecuencia a generar, recibir la señal analógica y guardar los resultados en un vector. Su diagrama de flujo se puede ver en la figura 3.6.

Los parámetros que recibe son: el objeto de conexión serial con el Arduino (que da acceso al buffer de transmisión y recepción); el vector de frecuencias a medir, los coeficientes del filtro FIR, un nombre para la medición (para identificar si es un barrido de calibración o de medición), y los errores de calibración si fuese necesario. Devuelve un vector de S11 complejos (formato binario).

Consiste de un bucle *for* que recorre el vector de frecuencias, seteando cada una en el Arduino con la función "write" de la librería "serial" para luego leer su respuesta con "readline". Si esta comienza en "error" aborta el barrido. De otra forma, captura el audio usando la librería "sounddevice". Para eso calcula el número de muestras a adquirir como la multiplicación de la frecuencia de muestreo y la duración de la captura. Graba los dos canales, de tipo "float64" con la función "rec". Descarta la primera parte de la grabación para evitar incluir en el cálculo los transitorios, y guarda dos vectores: "canal_ref" y "canal_med". Por último, mide S11 y la frecuencia en la que se encontró el batido. Para ir

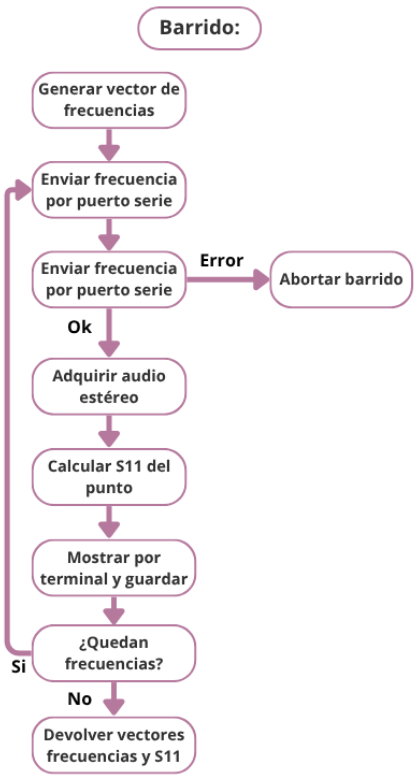


Figura 3.6: Diagrama de flujo de la función barrido.

mostrando los valores calibrados por terminal a medida que se hace el barrido, realiza la calibración por cada punto, calcula el S11 en módulo y fase y las imprime.

3.4.2 Estimación del S11

La función que realiza el cálculo del S11 tiene como parámetros de entrada los vectores de muestras, la frecuencia de muestreo y los coeficientes del filtro. Un diagrama de flujo simplificado se puede observar en la figura 3.7.

Lo primero que hace es aplicar el filtro a cada canal usando la función "lfilter" de scipy.signal, BLABLABLA. Luego calcula la FFT con la función "fft.fft()" de NumPy. Utiliza una ventana que setea el usuario, que por default es la FlatTop, y que crea la función "get_window" de scipy.signal.windows. Descarta la mitad negativa del espectro de frecuencias.

Luego busca el índice del bin frecuencial de mayor amplitud (en el canal de referencia) con la función "searchsorted" de NumPy dentro de un rango frecuencial acotado alrededor de f_{FI}, en la banda de paso del filtro.

$$k_{\mathrm{pico}} = \arg \max_{k \in \mathrm{Banda}} |X_{\mathrm{ref}}[k]| \tag{3.4.1}$$

Después crea un número infinitesimal basado en el pico del espectro para sumarselo al denominador del cálculo del S11, por si el valor de magnitud en la referencia es cero. Por último, calcula S11 como el conjugado de la división de la medición y la reflejada, usando para cada uno el bin donde se encontró el máximo de magnitud.

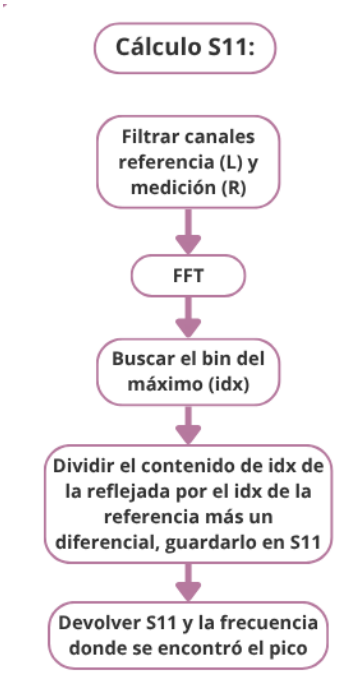


Figura 3.7: Diagrama de flujo de la función que calcula S11.

$$S_{11,\mathrm{raw}} = \frac{V_{\mathrm{med}}[k_{\mathrm{pico}}]}{V_{\mathrm{ref}}[k_{\mathrm{pico}}]} \tag{3.4.2}$$

Este número se almacena como un tipo complex128, un tipo de dato numérico que representa números complejos de doble precisión. Este valor no calibrado sirve como la entrada directa para el posterior algoritmo de corrección de errores vectoriales mediante el modelo de tres términos (SOL).

3.4.3 Calibración

El software guía al usuario para realizar tres barridos completos utilizando las cargas del método SOL. Almacena en tres vectores (GM_CC, GM_CA y GM_50) las mediciones complejas crudas del cortocircuito, circuito abierto y carga de 50 Ω respectivamente. Luego, la función "calculo_errores" procesa los datos punto a punto para cada frecuencia del barrido:

- Arma un sistema de ecuaciones lineales $A \cdot x = B$ con coeficientes complejos, donde A es la matriz cuya fila 1 corresponde a short: $[1, -\mathrm{GM_CC}[i], 1]$; fila 2 a load: 1; y fila 3 a open: $[1, \mathrm{GM_CA}[i], -1]$. El vector B corresponde a $[\mathrm{GM_CC}[i], \mathrm{GM_50}[i], \mathrm{GM_CA}[i]]$.
- Resuelve el sistema mediante la función "linalg.solve(A,B)" de NumPy, que permite obtener las tres incógnitas por cada punto en frecuencia.

Los tres vectores de error se guardan en un archivo CSV, separando las partes reales e imaginarias de cada término con notación científica de 10 dígitos decimales junto con la frecuencia en MHz.

Cuando se ejecuta el barrido, el software aplica la función de corrección inversa sobre cada punto medido según la ecuación 2.2.5. Antes de hacerlo, se verifica que el denominador no sea nulo. Si la magnitud es menor a 10^{-15}, se reemplaza por un valor insignificante ($10^{-15} + j10^{-15}$) para evitar errores de división por cero.

3.5 Interfaz de usuario

La interfaz de usuario se programó con PyQt6, mediante un prompt de la inteligencia artificial (IA) Claude, modelo Sonnet 5 Esfuerzo Medio. Se alimentó del software original, que tenía la lógica de medición funcionando, pero que corría secuencialmente y se comunicaba con el usuario a través de la consola.

El prompt inicial fue: "Quiero hacer una interfaz de usuario sencilla que me permita comunicarme con este script. que se pueda elegir frecuencias y paso, visualizar modulo y fase de S11 y abaco de smith, que permita hacer calibración. Qué plataforma/lenguaje recomendas, y cuanto tiempo tardaría en ponerla en funcionamiento?". La IA recomendó Python + PyQt6/PySide6, con matplotlib embebido para los gráficos. Dió las razones (reutilizar funciones, permite actualizar el plot en vivo, aprovecha librerías de Python para graficar) y explicó otras alternativas y por qué las descartó. Preguntó si hacía un esqueleto de la app con ventana principal, campos de frecuencia, threading de barrido y gráficos, y se le dió el permiso. Creó dos archivos, un "core" que contiene la misma lógica de cálculo que el código original pero reorganizada en funciones independientes (comunicación serial con Arduino, capura de audio, procesamiento de señal, detección del tono de batido y cálculo de S11 y álgebra de calibración) y un "GUI" (Graphical User Interface) que contiene la parte visual: ventana, botones, campos y gráficos. Este llama a las funciones implementadas en el core.

Se agregó la biblioteca PyQt6 para usar ventanas, botones, campos e hilos de ejecución. Se creó una clase (SweepWorker(QThread)) que se encarga de correr el barrido en segundo plano y avisar a la ventana qué se midió para que actualice los gráficos y la barra de progreso sin congelarse entre cada medición.

La IA generó además el diagrama de flujo que se observa en la figura 3.8. Los códigos se encuentran en el capítulo "Esquemáticos y código" 6.4.1.

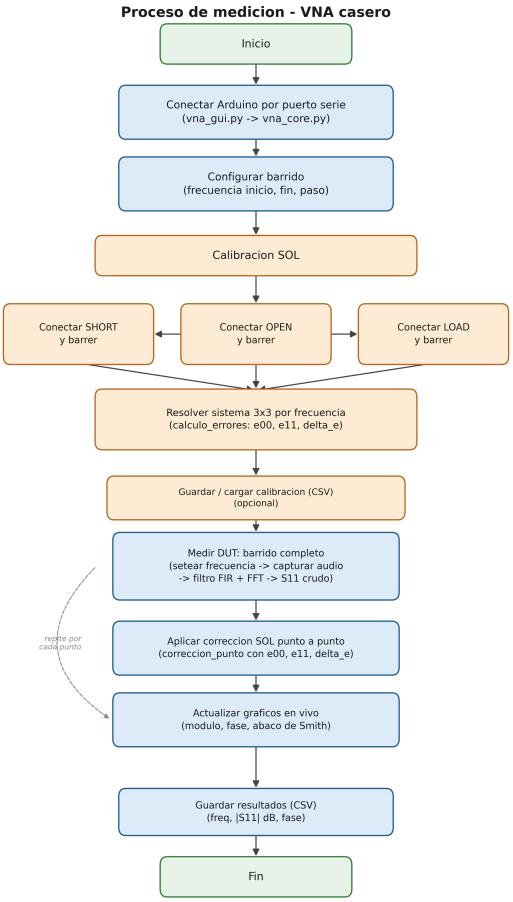


Figura 3.8: Diagrama de flujo de la interfaz de usuario

Capítulo 4

Resultados

Capítulo 5

Conclusiones y trabajos futuros

Capítulo 6

Esquemáticos y código

6.1 Esquemáticos Altium

A continuación se presentan los esquemáticos de los distintos bloques funcionales del sistema, exportados de Altium Designer.

6.2 Firmware

6.3 Software

6.4 Interfaz de usuario

6.4.1 Core

```
"""
vna_core.py
============
Nucleo de adquisicion y calculo para el VNA casero (2x ADF4351 + Arduino +
placa de sonido). Adaptado de VNA_calibrado.py para poder ser llamado desde
una interfaz grafica: sin input()/tkinter bloqueantes, con callback de
progreso por punto y posibilidad de abortar un barrido en curso.

La logica de senales (filtro FIR, FFT, deteccion del tono de batido,
algebra de calibracion SOL) es la misma que la del script original; lo
unico que cambia es como se invoca.

Requisitos:
    pip install pyserial sounddevice numpy scipy
"""

import csv
import time

import numpy as np
import serial
import sounddevice as sd
from scipy.signal import firwin, lfilter
from scipy.signal.windows import get_window


```

```
# ==================== Lista por defecto de frecuencias excluidas ====================
# (copiada de VNA_calibrado.py; se puede editar/reemplazar desde la GUI)

FRECUENCIAS_EXCLUIDAS_MHZ_DEFAULT = [
    200.00, 204.00, 208.00, 212.00, 216.00, 217.00, 221.00, 226.00, 256.00, 260.00, 280.00,
    285.00, 291.00, 320.00, 347.00, 353.00, 354.00, 359.00, 360.00, 365.00, 366.00, 371.00,
    372.00, 378.00, 379.00, 385.00, 386.00, 392.00, 393.00, 400.00, 401.00, 408.00, 409.00,
    416.00, 417.00, 424.00, 425.00, 433.00, 434.00, 443.00, 452.00, 512.00, 559.00, 560.00,
    570.00, 571.00, 581.00, 582.00, 617.00, 640.00, 678.00, 694.00, 705.00, 706.00, 707.00,
    708.00, 717.00, 718.00, 719.00, 720.00, 730.00, 731.00, 732.00, 742.00, 743.00, 744.00,
    745.00, 756.00, 757.00, 758.00, 759.00, 770.00, 771.00, 772.00, 773.00, 784.00, 785.00,
    786.00, 787.00, 799.00, 800.00, 801.00, 802.00, 815.00, 816.00, 817.00, 818.00, 831.00,
    832.00, 833.00, 834.00, 848.00, 849.00, 850.00, 851.00, 865.00, 866.00, 867.00, 868.00,
    883.00, 886.00, 904.00, 927.00, 929.00, 955.00, 1009.00, 1024.00, 1075.00, 1118.00, 1119.00,
    1120.00, 1140.00, 1142.00, 1162.00, 1164.00, 1185.00, 1187.00, 1234.00, 1280.00, 1283.00,
    1304.00, 1333.00, 1356.00, 1383.00, 1388.00, 1410.00, 1411.00, 1412.00, 1413.00, 1414.00,
    1415.00, 1416.00, 1434.00, 1435.00, 1436.00, 1437.00, 1438.00, 1439.00, 1440.00, 1459.00,
    1460.00, 1461.00, 1462.00, 1463.00, 1464.00, 1465.00, 1484.00, 1485.00, 1486.00, 1487.00,
    1488.00, 1489.00, 1490.00, 1491.00, 1511.00, 1512.00, 1513.00, 1514.00, 1515.00, 1516.00,
    1517.00, 1518.00, 1539.00, 1540.00, 1541.00, 1542.00, 1543.00, 1544.00, 1545.00, 1546.00,
    1568.00, 1569.00, 1570.00, 1571.00, 1572.00, 1573.00, 1574.00, 1575.00, 1598.00, 1599.00,
    1600.00, 1601.00, 1602.00, 1603.00, 1604.00, 1605.00, 1629.00, 1630.00, 1631.00, 1632.00,
    1633.00, 1634.00, 1635.00, 1636.00, 1662.00, 1663.00, 1664.00, 1665.00, 1666.00, 1667.00,
    1668.00, 1669.00, 1696.00, 1697.00, 1698.00, 1700.00, 1701.00, 1702.00, 1730.00, 1731.00,
    1732.00, 1734.00, 1735.00, 1736.00, 1766.00, 1767.00, 1771.00, 1772.00, 1803.00, 1829.00,
    1833.00, 1854.00, 1858.00, 1910.00, 1937.00, 2018.00, 2150.00, 2235.00, 2236.00,
    2237.00, 2238.00, 2239.00, 2240.00, 2241.00, 2279.00, 2280.00, 2281.00, 2283.00, 2284.00,
```

```
    2324.00, 2325.00, 2327.00, 2328.00, 2329.00, 2370.00, 2374.00, 2421.00, 2463.00,
]


# ==================== Configuracion por defecto ====================

def config_por_defecto():
    """Devuelve un dict de configuracion editable desde la GUI."""
    return {
        "fs_audio": 44100,
        "duracion_captura_s": 0.6,
        "recorte_inicial_s": 0.150,
        "dispositivo_audio": None,
        "f_centro_filtro": 10000,      # Hz, tono de batido esperado
        "ancho_banda_filtro": 200,     # Hz
        "numtaps_filtro": 1001,
        "ventana_fft": "flattop",
        "frecuencias_excluidas": list(FRECUENCIAS_EXCLUIDAS_MHZ_DEFAULT),
    }


def banda_busqueda_pico(config):
    fc = config["f_centro_filtro"]
    bw = config["ancho_banda_filtro"]
    return (fc - bw / 2, fc + bw / 2)


# ==================== Arduino / serial ====================

class ErrorArduino(Exception):
    pass


def conectar_arduino(puerto, baudrate=115200, timeout=5):
    """Abre el puerto serie y devuelve (objeto_serial, primera_linea_recibida)."""
    ser = serial.Serial(puerto, baudrate, timeout=timeout)
    time.sleep(2)  # el Arduino se resetea al abrir el puerto; esperamos a que arranque
    ser.reset_input_buffer()
    linea = ser.readline().decode(errors="ignore").strip()
    return ser, linea


def setear_frecuencia(ser, f_mhz):
    """Manda la frecuencia al Arduino y espera la confirmacion (OK/ERROR)."""
    comando = f"{f_mhz:.6f}\n"
    ser.write(comando.encode())
    respuesta = ser.readline().decode(errors="ignore").strip()
    intentos_vacios = 0
```

```
    while respuesta and not respuesta.startswith("OK") and not
    ↪ respuesta.startswith("ERROR"):
        respuesta = ser.readline().decode(errors="ignore").strip()
        intentos_vacios += 1
        if intentos_vacios > 50:
            break
    return respuesta


def generar_frecuencias(inicio, fin, paso):
    """Genera la lista de frecuencias evitando arrastre de error de punto
    ↪ flotante."""
    if paso <= 0:
        raise ValueError("El paso de frecuencia debe ser mayor que 0.")
    n_pasos = round((fin - inicio) / paso)
    return [inicio + i * paso for i in range(n_pasos + 1)]


# ==================== Adquisicion de audio ====================

def capturar_audio(duracion_s, fs, dispositivo=None, recorte_inicial_s=0.150):
    """
    Graba desde la entrada configurada. Devuelve (canal_ref, canal_med) como
    arrays float64, con canal_ref = izquierdo, canal_med = derecho.
    """
    n_muestras = int(duracion_s * fs)
    grabacion_con_ruido = sd.rec(
        n_muestras, samplerate=fs, channels=2, dtype="float64",
        ↪ device=dispositivo
    )
    sd.wait()
    grabacion = grabacion_con_ruido[int(fs * recorte_inicial_s):]
    canal_ref = grabacion[:, 0]
    canal_med = grabacion[:, 1]
    return canal_ref, canal_med


# ==================== Filtro FIR + FFT + S11 ====================

def diseñar_filtro_pasabanda(fs, f_centro, ancho_banda, numtaps,
↪ duracion_captura_s):
    n_muestras_captura = int(duracion_captura_s * fs)
    if numtaps >= n_muestras_captura:
        raise ValueError(
            f"NUMTAPS_FILTRO ({numtaps}) debe ser menor que la cantidad de "
            f"muestras por captura ({n_muestras_captura}, con
            ↪ duracion_captura_s="
            f"{duracion_captura_s}s y fs={fs}Hz). Bajar numtaps o subir la
            ↪ duracion."
        )
    f_baja = f_centro - ancho_banda / 2
    f_alta = f_centro + ancho_banda / 2
```

```
148     return firwin(numtaps, [f_baja, f_alta], pass_zero=False, fs=fs,
        ↪  window="hamming")
149
150
151 def aplicar_filtro(señal, coeficientes):
152     return lfilter(coeficientes, 1.0, señal)
153
154
155 def calcular_espectro_complejo(señal, fs, ventana):
156     n = len(señal)
157     w = get_window(ventana, n)
158     fft_vals = np.fft.fft(señal * w)
159     fft_freqs = np.fft.fftfreq(n, d=1 / fs)
160     mitad = n // 2
161     return fft_freqs[:mitad], fft_vals[:mitad]
162
163
164 def buscar_indice_pico(freqs, espectro, banda):
165     """Devuelve el indice del bin de mayor magnitud dentro de la banda dada."""
166     f_baja, f_alta = banda
167     i0 = np.searchsorted(freqs, f_baja)
168     i1 = np.searchsorted(freqs, f_alta)
169     if i1 <= i0:
170         raise ValueError("La banda de busqueda no contiene ningun bin de
            ↪  frecuencia.")
171     magnitud = np.abs(espectro[i0:i1])
172     return i0 + np.argmax(magnitud)
173
174
175 def medir_s11_punto(canal_ref, canal_med, fs, coef_filtro, ventana_fft, banda):
176     """
177     A partir de una captura de referencia y medicion, devuelve un unico
178     valor complejo de S11 (sin corregir) correspondiente al tono de batido,
179     y la frecuencia de batido detectada (Hz).
180     """
181     ref_filtrada = aplicar_filtro(canal_ref, coef_filtro)
182     med_filtrada = aplicar_filtro(canal_med, coef_filtro)
183
184     freqs_ref, espectro_ref = calcular_espectro_complejo(ref_filtrada, fs,
        ↪  ventana_fft)
185     _, espectro_med = calcular_espectro_complejo(med_filtrada, fs, ventana_fft)
186
187     idx_pico = buscar_indice_pico(freqs_ref, espectro_ref, banda)
188
189     epsilon = np.max(np.abs(espectro_ref)) * 1e-9
190     s11 = np.conj(espectro_med[idx_pico] / (espectro_ref[idx_pico] + epsilon))
191
192     return s11, freqs_ref[idx_pico]
193
194
195 # ==================== Barrido (con callback de progreso + aborto)
    ↪  ====================
```

```python

class BarridoAbortado(Exception):
    """Se lanza cuando should_abort() devuelve True durante un barrido."""
    pass


def barrido(ser, frecuencias_rf, coef_filtro, config, on_point=None,
    should_abort=None):
    """
    Ejecuta un barrido completo de frecuencias, midiendo un punto de S11
    (sin corregir) por cada frecuencia.

    on_point(i, n, f_rf, s11, f_batido_hz): callback opcional, llamado
        despues de medir cada punto (i es 1-based).
    should_abort(): callable opcional; si devuelve True se aborta el
        barrido lanzando BarridoAbortado.

    Devuelve (freqs_medidas, s11_medidos) como arrays de numpy, salteando
    las frecuencias que esten en config["frecuencias_excluidas"].
    """
    resultados_freq_rf = []
    resultados_s11 = []
    excluidas = config.get("frecuencias_excluidas", [])
    banda = banda_busqueda_pico(config)

    for i, f_rf in enumerate(frecuencias_rf):
        if should_abort is not None and should_abort():
            raise BarridoAbortado()

        if any(abs(f_rf - fx) < 1e-6 for fx in excluidas):
            continue

        respuesta = setear_frecuencia(ser, f_rf)
        if respuesta.startswith("ERROR"):
            raise ErrorArduino(f"Arduino devolvio error en {f_rf:.3f} MHz:
            {respuesta}")

        canal_ref, canal_med = capturar_audio(
            config["duracion_captura_s"], config["fs_audio"],
            config.get("dispositivo_audio"),
            config.get("recorte_inicial_s", 0.150),
        )
        s11, f_batido = medir_s11_punto(
            canal_ref, canal_med, config["fs_audio"], coef_filtro,
            config["ventana_fft"], banda,
        )

        resultados_freq_rf.append(f_rf)
        resultados_s11.append(s11)

        if on_point is not None:
            on_point(i + 1, len(frecuencias_rf), f_rf, s11, f_batido)
```

```
245
246     return np.array(resultados_freq_rf), np.array(resultados_s11)
247
248
249 # ==================== Calibracion SOL ====================
250
251 def calculo_errores(GM_CC, GM_CA, GM_50, n):
252     """Resuelve, punto a punto, el sistema 3x3 de la calibracion SOL."""
253     e_00 = np.empty(n, dtype=complex)
254     e_11 = np.empty(n, dtype=complex)
255     delta_e = np.empty(n, dtype=complex)
256
257     for i in range(n):
258         A = np.array([
259             [1, -GM_CC[i], 1],  # CC (short)
260             [1, 0, 0],          # 50 (load)
261             [1, GM_CA[i], -1],  # CA (open)
262         ], dtype=complex)
263         B = np.array([GM_CC[i], GM_50[i], GM_CA[i]])
264         sol = np.linalg.solve(A, B)
265         e_00[i] = sol[0]
266         e_11[i] = sol[1]
267         delta_e[i] = sol[2]
268
269     return e_00, e_11, delta_e
270
271
272 def correccion(GM, e_00, e_11, delta_e, n):
273     """Aplica la correccion de error de la calibracion SOL a un array de GM
    ↪  medidos."""
274     Gamma = np.empty(n, dtype=complex)
275     for i in range(n):
276         denominador = (GM[i] * e_11[i] - delta_e[i])
277         if abs(denominador) < 1e-15:
278             denominador = 1e-15 + 1e-15j
279         Gamma[i] = (GM[i] - e_00[i]) / denominador
280     return Gamma
281
282
283 def correccion_punto(gm, e_00_i, e_11_i, delta_e_i):
284     """Version de correccion() para un unico punto (usada para graficar en
    ↪  vivo)."""
285     denominador = gm * e_11_i - delta_e_i
286     if abs(denominador) < 1e-15:
287         denominador = 1e-15 + 1e-15j
288     return (gm - e_00_i) / denominador
289
290
291 def guardar_calibracion_csv(path, freqs, e_00, e_11, delta_e):
292     with open(path, "w", newline="", encoding="utf-8") as f:
293         writer = csv.writer(f)
```

```
        writer.writerow(["freq_MHz", "e_00_real", "e_00_imag", "e_11_real", "e_11_imag",
                         "delta_e_real", "delta_e_imag"])
        for i in range(len(freqs)):
            writer.writerow([
                f"{freqs[i]:.6f}",
                f"{e_00[i].real:.10e}", f"{e_00[i].imag:.10e}",
                f"{e_11[i].real:.10e}", f"{e_11[i].imag:.10e}",
                f"{delta_e[i].real:.10e}", f"{delta_e[i].imag:.10e}",
            ])


def cargar_calibracion_csv(path):
    """Rearma los valores complejos guardados por guardar_calibracion_csv()."""
    freqs, e_00, e_11, delta_e = [], [], [], []
    with open(path, "r", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            freqs.append(float(row["freq_MHz"]))
            e_00.append(complex(float(row["e_00_real"]), float(row["e_00_imag"])))
            e_11.append(complex(float(row["e_11_real"]), float(row["e_11_imag"])))
            delta_e.append(complex(float(row["delta_e_real"]), float(row["delta_e_imag"])))
    return np.array(freqs), np.array(e_00), np.array(e_11), np.array(delta_e)


def guardar_resultados_csv(path, freq_rf, modulo_db, fase_deg):
    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["freq_rf_MHz", "modulo_S11_dB", "fase_S11_deg"])
        for f_rf, m, fa in zip(freq_rf, modulo_db, fase_deg):
            writer.writerow([f"{f_rf:.6f}", f"{m:.4f}", f"{fa:.4f}"])


def modulo_fase(s11_array):
    """Convierte un array de S11 complejos a (modulo_dB, fase_grados)."""
    modulo_db = 20 * np.log10(np.abs(s11_array) + 1e-12)
    fase_deg = np.degrees(np.angle(s11_array))
    return modulo_db, fase_deg
```

6.4.2 Graphical User Interface

```
"""
vna_gui.py
==========
Interfaz grafica (PyQt6) para el VNA casero: permite elegir frecuencia de
inicio/fin/paso, conectar el Arduino, correr la calibracion SOL (short /
open / load) paso a paso, medir un DUT, y visualizar en vivo el modulo y
la fase de S11 junto con el abaco de Smith.

Requisitos:
```

```
    pip install PyQt6 pyserial sounddevice numpy scipy matplotlib

Ejecutar con:
    python vna_gui.py
"""

import os
import sys

import numpy as np

from PyQt6.QtCore import Qt, QThread, pyqtSignal
from PyQt6.QtWidgets import (
    QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
    QGroupBox, QLabel, QLineEdit, QPushButton, QDoubleSpinBox,
    QProgressBar, QFileDialog, QMessageBox, QPlainTextEdit, QSplitter,
)

from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg as FigureCanvas
from matplotlib.figure import Figure

import vna_core as core


# ==================== Hilo de trabajo: corre un barrido sin trabar la GUI ====================

class SweepWorker(QThread):
    """Ejecuta core.barrido() en un hilo aparte y reenvia sus resultados como señales Qt."""

    punto_medido = pyqtSignal(int, int, float, complex, float)  # i, n, f_rf, s11, f_batido
    terminado = pyqtSignal(object, object)                      # freqs (ndarray), s11 (ndarray)
    abortado = pyqtSignal()
    error = pyqtSignal(str)

    def __init__(self, ser, frecuencias_rf, coef_filtro, config, parent=None):
        super().__init__(parent)
        self.ser = ser
        self.frecuencias_rf = frecuencias_rf
        self.coef_filtro = coef_filtro
        self.config = config
        self._abortar = False

    def abortar(self):
        self._abortar = True

    def run(self):
        try:
            freqs, s11 = core.barrido(
```

```
58                  self.ser, self.frecuencias_rf, self.coef_filtro, self.config,
59                  on_point=lambda i, n, f, s, fb: self.punto_medido.emit(i, n, f,
                    ↪  s, fb),
60                  should_abort=lambda: self._abortar,
61              )
62              self.terminado.emit(freqs, s11)
63          except core.BarridoAbortado:
64              self.abortado.emit()
65          except Exception as e:  # noqa: BLE001 - se muestra en la GUI
66              self.error.emit(str(e))
67
68
69  # ==================== Dibujo del abaco de Smith (fondo estatico)
    ↪  ====================
70
71  def dibujar_fondo_smith(ax):
72      """Dibuja el grillado clasico de un abaco de Smith (r y x constantes)."""
73      ax.clear()
74      ax.set_aspect("equal")
75      ax.axis("off")
76      ax.set_xlim(-1.15, 1.15)
77      ax.set_ylim(-1.15, 1.15)
78      ax.set_title("Abaco de Smith")
79
80      theta = np.linspace(0, 2 * np.pi, 400)
81
82      # Circulo unitario (|Gamma| = 1)
83      ax.plot(np.cos(theta), np.sin(theta), color="black", linewidth=1.0)
84      ax.axhline(0, color="gray", linewidth=0.5)
85
86      # Circulos de resistencia constante (r = 0, 0.2, 0.5, 1, 2, 5)
87      for r in (0.0, 0.2, 0.5, 1.0, 2.0, 5.0):
88          cx = r / (r + 1)
89          rad = 1 / (r + 1)
90          x = cx + rad * np.cos(theta)
91          y = rad * np.sin(theta)
92          ax.plot(x, y, color="gray", linewidth=0.5)
93
94      # Arcos de reactancia constante (x = +/- 0.2, 0.5, 1, 2, 5)
95      t = np.linspace(0, 2 * np.pi, 800)
96      for x_val in (0.2, 0.5, 1.0, 2.0, 5.0):
97          for signo in (1, -1):
98              xv = signo * x_val
99              cx, cy = 1.0, 1.0 / xv
100             rad = abs(1.0 / xv)
101             gx = cx + rad * np.cos(t)
102             gy = cy + rad * np.sin(t)
103             mask = gx ** 2 + gy ** 2 <= 1.0 + 1e-6
104             ax.plot(gx[mask], gy[mask], color="gray", linewidth=0.5)
105
106
107 # ==================== Ventana principal ====================
```

```
108
109 class MainWindow(QMainWindow):
110     def __init__(self):
111         super().__init__()
112         self.setWindowTitle("VNA - Control y medición de S11")
113         self.resize(1200, 750)
114
115         self.config = core.config_por_defecto()
116         self.ser = None
117         self.hilo = None
118
119         # Estado de calibracion
120         self.cal_paso = 0            # 0 = sin empezar, 1=SHORT, 2=OPEN, 3=LOAD,
            ↪  4=lista
121         self.cal_gm_cc = None
122         self.cal_gm_ca = None
123         self.cal_gm_50 = None
124         self.cal_freqs = None
125         self.cal_e00 = None
126         self.cal_e11 = None
127         self.cal_delta_e = None
128         self.cal_disponible = False
129
130         # Buffers de la medicion en curso (para graficar en vivo)
131         self.buf_freq = []
132         self.buf_mod_db = []
133         self.buf_fase_deg = []
134         self.buf_gamma = []
135         self.ultima_medicion_freqs = None
136         self.ultima_medicion_gamma = None
137         self.ultima_medicion_mod_db = None
138         self.ultima_medicion_fase_deg = None
139
140         self._armar_ui()
141
142     # ------------------------------------------------------------------ UI --
143
144     def _armar_ui(self):
145         central = QWidget()
146         self.setCentralWidget(central)
147         layout_principal = QHBoxLayout(central)
148
149         splitter = QSplitter(Qt.Orientation.Horizontal)
150         layout_principal.addWidget(splitter)
151
152         panel_izq = self._armar_panel_controles()
153         panel_der = self._armar_panel_graficos()
154
155         splitter.addWidget(panel_izq)
156         splitter.addWidget(panel_der)
157         splitter.setStretchFactor(0, 0)
158         splitter.setStretchFactor(1, 1)
```

```
159
160     def _armar_panel_controles(self):
161         panel = QWidget()
162         panel.setMinimumWidth(340)
163         panel.setMaximumWidth(420)
164         v = QVBoxLayout(panel)
165
166         # --- Conexion Arduino ---
167         gb_conexion = QGroupBox("Conexion Arduino")
168         f = QVBoxLayout(gb_conexion)
169         fila_puerto = QHBoxLayout()
170         fila_puerto.addWidget(QLabel("Puerto:"))
171         self.ed_puerto = QLineEdit("COM3")
172         fila_puerto.addWidget(self.ed_puerto)
173         f.addLayout(fila_puerto)
174
175         fila_baud = QHBoxLayout()
176         fila_baud.addWidget(QLabel("Baudrate:"))
177         self.ed_baudrate = QLineEdit("115200")
178         fila_baud.addWidget(self.ed_baudrate)
179         f.addLayout(fila_baud)
180
181         self.btn_conectar = QPushButton("Conectar")
182         self.btn_conectar.clicked.connect(self._conectar_arduino)
183         f.addWidget(self.btn_conectar)
184
185         self.lbl_estado_conexion = QLabel("Desconectado")
186         f.addWidget(self.lbl_estado_conexion)
187         v.addWidget(gb_conexion)
188
189         # --- Barrido ---
190         gb_barrido = QGroupBox("Barrido de frecuencia")
191         f2 = QVBoxLayout(gb_barrido)
192
193         self.sp_f_inicio = self._nuevo_spin(1800.0, 0, 6000, " MHz")
194         self.sp_f_fin = self._nuevo_spin(2200.0, 0, 6000, " MHz")
195         self.sp_f_paso = self._nuevo_spin(1.0, 0.001, 1000, " MHz")
196
197         f2.addWidget(self._fila("F inicio:", self.sp_f_inicio))
198         f2.addWidget(self._fila("F fin:", self.sp_f_fin))
199         f2.addWidget(self._fila("F paso:", self.sp_f_paso))
200         v.addWidget(gb_barrido)
201
202         # --- Calibracion SOL ---
203         gb_cal = QGroupBox("Calibracion SOL")
204         f3 = QVBoxLayout(gb_cal)
205
206         self.lbl_estado_cal = QLabel("Sin calibrar")
207         f3.addWidget(self.lbl_estado_cal)
208
209         self.btn_cal_iniciar = QPushButton("Iniciar calibracion nueva")
210         self.btn_cal_iniciar.clicked.connect(self._iniciar_calibracion)
```

```
211         f3.addWidget(self.btn_cal_iniciar)
212
213         self.btn_cal_paso = QPushButton("Barrer paso actual")
214         self.btn_cal_paso.setEnabled(False)
215         self.btn_cal_paso.clicked.connect(self._barrer_paso_calibracion)
216         f3.addWidget(self.btn_cal_paso)
217
218         self.btn_cal_cargar = QPushButton("Cargar calibracion desde archivo...")
219         self.btn_cal_cargar.clicked.connect(self._cargar_calibracion)
220         f3.addWidget(self.btn_cal_cargar)
221
222         self.btn_cal_guardar = QPushButton("Guardar calibracion actual...")
223         self.btn_cal_guardar.setEnabled(False)
224         self.btn_cal_guardar.clicked.connect(self._guardar_calibracion)
225         f3.addWidget(self.btn_cal_guardar)
226
227         v.addWidget(gb_cal)
228
229         # --- Medicion DUT ---
230         gb_dut = QGroupBox("Medición del DUT")
231         f4 = QVBoxLayout(gb_dut)
232
233         self.btn_medir = QPushButton("Medir DUT")
234         self.btn_medir.setEnabled(False)
235         self.btn_medir.clicked.connect(self._iniciar_medicion_dut)
236         f4.addWidget(self.btn_medir)
237
238         self.btn_abortar = QPushButton("Abortar barrido")
239         self.btn_abortar.setEnabled(False)
240         self.btn_abortar.clicked.connect(self._abortar_barrido)
241         f4.addWidget(self.btn_abortar)
242
243         self.progreso = QProgressBar()
244         f4.addWidget(self.progreso)
245
246         self.btn_guardar_resultados = QPushButton("Guardar resultados CSV...")
247         self.btn_guardar_resultados.setEnabled(False)
248         self.btn_guardar_resultados.clicked.connect(self._guardar_resultados)
249         f4.addWidget(self.btn_guardar_resultados)
250
251         v.addWidget(gb_dut)
252
253         # --- Log ---
254         gb_log = QGroupBox("Log")
255         f5 = QVBoxLayout(gb_log)
256         self.txt_log = QPlainTextEdit()
257         self.txt_log.setReadOnly(True)
258         f5.addWidget(self.txt_log)
259         v.addWidget(gb_log, stretch=1)
260
261         return panel
262
```

```
263     def _armar_panel_graficos(self):
264         panel = QWidget()
265         v = QVBoxLayout(panel)
266 
267         # Figura módulo + fase (dos subplots apilados)
268         self.fig_mf = Figure(figsize=(6, 5), facecolor="#fad7ea")
269         self.ax_mod = self.fig_mf.add_subplot(211)
270         self.ax_fase = self.fig_mf.add_subplot(212, sharex=self.ax_mod)
271         self._preparar_ejes_mod_fase()
272         self.canvas_mf = FigureCanvas(self.fig_mf)
273 
274         # Figura abaco de Smith
275         self.fig_smith = Figure(figsize=(5, 5), facecolor="#fad7ea")
276         self.ax_smith = self.fig_smith.add_subplot(111)
277         dibujar_fondo_smith(self.ax_smith)
278         (self.linea_smith,) = self.ax_smith.plot([], [], color="deeppink",
279                                                  marker=".", linestyle="None")
280         self.canvas_smith = FigureCanvas(self.fig_smith)
281 
282         fila_graficos = QHBoxLayout()
283         fila_graficos.addWidget(self.canvas_mf, stretch=1)
284         fila_graficos.addWidget(self.canvas_smith, stretch=1)
285         v.addLayout(fila_graficos)
286 
287         return panel
288 
289     def _preparar_ejes_mod_fase(self):
290         self.ax_mod.clear()
291         self.ax_mod.set_facecolor("#fad7ea")
292         self.ax_mod.set_title("S11 - Módulo")
293         self.ax_mod.set_ylabel("|S11| (dB)")
294         self.ax_mod.grid(True, alpha=0.3)
295         (self.linea_mod,) = self.ax_mod.plot([], [], color="deeppink",
296                                              marker=".", linestyle="None")
297 
298         self.ax_fase.clear()
299         self.ax_fase.set_facecolor("#fad7ea")
300         self.ax_fase.set_title("S11 - Fase")
301         self.ax_fase.set_xlabel("Frecuencia de RF (MHz)")
302         self.ax_fase.set_ylabel("Fase (grados)")
303         self.ax_fase.grid(True, alpha=0.3)
304         (self.linea_fase,) = self.ax_fase.plot([], [], color="darkmagenta",
305                                                marker=".", linestyle="None")
306         self.fig_mf.tight_layout()
307 
308     @staticmethod
309     def _nuevo_spin(valor, minimo, maximo, sufijo):
310         sp = QDoubleSpinBox()
311         sp.setDecimals(3)
312         sp.setRange(minimo, maximo)
313         sp.setValue(valor)
314         sp.setSuffix(sufijo)
```

```
315         return sp
316
317     @staticmethod
318     def _fila(etiqueta, widget):
319         w = QWidget()
320         h = QHBoxLayout(w)
321         h.setContentsMargins(0, 0, 0, 0)
322         h.addWidget(QLabel(etiqueta))
323         h.addWidget(widget)
324         return w
325
326     def _log(self, texto):
327         self.txt_log.appendPlainText(texto)
328
329     # ------------------------------------------------------------ Conexion --
330
331     def _conectar_arduino(self):
332         puerto = self.ed_puerto.text().strip()
333         try:
334             baudrate = int(self.ed_baudrate.text().strip())
335         except ValueError:
336             QMessageBox.warning(self, "Baudrate invalido", "El baudrate debe ser
                ↪  un numero entero.")
337             return
338
339         try:
340             self.ser, linea = core.conectar_arduino(puerto, baudrate)
341         except Exception as e:
342             QMessageBox.critical(self, "Error de conexion", str(e))
343             self.lbl_estado_conexion.setText("Error de conexion")
344             return
345
346         self.lbl_estado_conexion.setText(f"Conectado ({puerto} @ {baudrate})")
347         if linea:
348             self._log(f"Arduino: {linea}")
349         self._log(f"Conectado a {puerto} @ {baudrate} baud.")
350
351     # ------------------------------------------------------------ Utilidades --
352
353     def _construir_frecuencias(self):
354         inicio = self.sp_f_inicio.value()
355         fin = self.sp_f_fin.value()
356         paso = self.sp_f_paso.value()
357         return core.generar_frecuencias(inicio, fin, paso)
358
359     def _construir_filtro(self):
360         return core.diseñar_filtro_pasabanda(
361             self.config["fs_audio"], self.config["f_centro_filtro"],
362             self.config["ancho_banda_filtro"], self.config["numtaps_filtro"],
363             self.config["duracion_captura_s"],
364         )
365
```

```
366     def _validar_listo_para_barrer(self):
367         if self.ser is None:
368             QMessageBox.warning(self, "Sin conexion", "Primero conecta el
                ↪ Arduino.")
369             return False
370         if self.hilo is not None and self.hilo.isRunning():
371             QMessageBox.warning(self, "Barrido en curso", "Ya hay un barrido
                ↪ corriendo.")
372             return False
373         return True
374 
375     def _lanzar_barrido(self, frecuencias, on_terminado, on_abortado=None,
    ↪ on_error=None):
376         coef_filtro = self._construir_filtro()
377         self.hilo = SweepWorker(self.ser, frecuencias, coef_filtro, self.config)
378         self.hilo.punto_medido.connect(self._on_punto_generico)
379         self.hilo.terminado.connect(on_terminado)
380         self.hilo.abortado.connect(on_abortado or self._on_abortado_generico)
381         self.hilo.error.connect(on_error or self._on_error_generico)
382         self.progreso.setValue(0)
383         self.btn_abortar.setEnabled(True)
384         self.hilo.start()
385 
386     def _on_punto_generico(self, i, n, f_rf, s11, f_batido):
387         self.progreso.setMaximum(n)
388         self.progreso.setValue(i)
389         mod_db = 20 * np.log10(abs(s11) + 1e-12)
390         fase_deg = np.degrees(np.angle(s11))
391         self._log(f"[{i}/{n}] RF={f_rf:.3f} MHz  batido={f_batido:.1f} Hz  "
392                   f"|S11|={mod_db:.2f} dB  fase={fase_deg:.1f} deg")
393 
394     def _on_abortado_generico(self):
395         self.btn_abortar.setEnabled(False)
396         self._log("Barrido abortado por el usuario.")
397 
398     def _on_error_generico(self, mensaje):
399         self.btn_abortar.setEnabled(False)
400         QMessageBox.critical(self, "Error durante el barrido", mensaje)
401         self._log(f"ERROR: {mensaje}")
402 
403     def _abortar_barrido(self):
404         if self.hilo is not None and self.hilo.isRunning():
405             self.hilo.abortar()
406 
407     # ------------------------------------------------------- Calibracion --
408 
409     ETIQUETAS_PASO_CAL = {
410         1: "Paso 1/3: conecte SHORT (cortocircuito) y presione 'Barrer paso
            ↪ actual'.",
411         2: "Paso 2/3: conecte OPEN (circuito abierto) y presione 'Barrer paso
            ↪ actual'.",
412         3: "Paso 3/3: conecte LOAD (50 ohm) y presione 'Barrer paso actual'.",
```

```
413         }
414
415     def _iniciar_calibracion(self):
416         if not self._validar_listo_para_barrer():
417             return
418         self.cal_paso = 1
419         self.cal_gm_cc = self.cal_gm_ca = self.cal_gm_50 = None
420         self.cal_disponible = False
421         self.btn_medir.setEnabled(False)
422         self.btn_cal_guardar.setEnabled(False)
423         self.btn_cal_paso.setEnabled(True)
424         self.lbl_estado_cal.setText(self.ETIQUETAS_PASO_CAL[1])
425
426     def _barrer_paso_calibracion(self):
427         if not self._validar_listo_para_barrer():
428             return
429         frecuencias = self._construir_frecuencias()
430         self.btn_cal_paso.setEnabled(False)
431         self.btn_cal_iniciar.setEnabled(False)
432         self._lanzar_barrido(
433             frecuencias,
434             on_terminado=self._on_paso_calibracion_terminado,
435             on_abortado=self._on_calibracion_abortada,
436         )
437
438     def _on_calibracion_abortada(self):
439         self._on_abortado_generico()
440         self.btn_cal_paso.setEnabled(True)
441         self.btn_cal_iniciar.setEnabled(True)
442
443     def _on_paso_calibracion_terminado(self, freqs, gm):
444         self.btn_abortar.setEnabled(False)
445         self.btn_cal_iniciar.setEnabled(True)
446         self.cal_freqs = freqs
447
448         if self.cal_paso == 1:
449             self.cal_gm_cc = gm
450             self.cal_paso = 2
451             self.lbl_estado_cal.setText(self.ETIQUETAS_PASO_CAL[2])
452             self.btn_cal_paso.setEnabled(True)
453         elif self.cal_paso == 2:
454             self.cal_gm_ca = gm
455             self.cal_paso = 3
456             self.lbl_estado_cal.setText(self.ETIQUETAS_PASO_CAL[3])
457             self.btn_cal_paso.setEnabled(True)
458         elif self.cal_paso == 3:
459             self.cal_gm_50 = gm
460             self._finalizar_calculo_calibracion()
461
462     def _finalizar_calculo_calibracion(self):
463         n = len(self.cal_freqs)
```

```
464             if not (len(self.cal_gm_cc) == len(self.cal_gm_ca) ==
                ↪  len(self.cal_gm_50) == n):
465                 QMessageBox.warning(
466                     self, "Inconsistencia",
467                     "Los tres barridos de calibracion (SHORT/OPEN/LOAD) no tienen la
                    ↪  "
468                     "misma cantidad de puntos. Revisa que no se haya abortado
                    ↪  ninguno "
469                     "a mitad de camino."
470                 )
471                 self.cal_paso = 0
472                 self.btn_cal_paso.setEnabled(False)
473                 return
474 
475             self.cal_e00, self.cal_e11, self.cal_delta_e = core.calculo_errores(
476                 self.cal_gm_cc, self.cal_gm_ca, self.cal_gm_50, n
477             )
478             self.cal_disponible = True
479             self.cal_paso = 4
480             self.lbl_estado_cal.setText(f"Calibracion lista ({n} puntos).")
481             self.btn_cal_paso.setEnabled(False)
482             self.btn_cal_guardar.setEnabled(True)
483             self.btn_medir.setEnabled(True)
484             self._log("Calibracion SOL calculada correctamente.")
485 
486         def _cargar_calibracion(self):
487             path, _ = QFileDialog.getOpenFileName(
488                 self, "Cargar calibracion", "", "CSV (*.csv)"
489             )
490             if not path:
491                 return
492             try:
493                 freqs, e00, e11, delta_e = core.cargar_calibracion_csv(path)
494             except Exception as e:
495                 QMessageBox.critical(self, "Error al cargar", str(e))
496                 return
497 
498             self.cal_freqs, self.cal_e00, self.cal_e11, self.cal_delta_e = freqs,
                ↪  e00, e11, delta_e
499             self.cal_disponible = True
500             self.cal_paso = 4
501             self.lbl_estado_cal.setText(f"Calibracion cargada desde archivo
                ↪  ({len(freqs)} puntos).")
502             self.btn_medir.setEnabled(True)
503             self.btn_cal_guardar.setEnabled(False)
504             self._log(f"Calibracion cargada de {path}")
505 
506         def _guardar_calibracion(self):
507             if not self.cal_disponible:
508                 return
509             path, _ = QFileDialog.getSaveFileName(
510                 self, "Guardar calibracion", "calibracion.csv", "CSV (*.csv)"
```

```
511         )
512         if not path:
513             return
514         core.guardar_calibracion_csv(path, self.cal_freqs, self.cal_e00,
            ↪  self.cal_e11, self.cal_delta_e)
515         self._log(f"Calibracion guardada en {path}")
516 
517     # ------------------------------------------------------ Medicion DUT --
518 
519     def _iniciar_medicion_dut(self):
520         if not self._validar_listo_para_barrer():
521             return
522         if not self.cal_disponible:
523             QMessageBox.warning(self, "Sin calibracion", "Primero completa o
                ↪  carga una calibracion.")
524             return
525 
526         frecuencias = self._construir_frecuencias()
527 
528         # Reiniciar buffers y graficos
529         self.buf_freq, self.buf_mod_db, self.buf_fase_deg, self.buf_gamma = [],
            ↪  [], [], []
530         self._preparar_ejes_mod_fase()
531         self.canvas_mf.draw()
532         dibujar_fondo_smith(self.ax_smith)
533         (self.linea_smith,) = self.ax_smith.plot([], [], color="deeppink",
            ↪  marker=".", linestyle="None")
534         self.canvas_smith.draw()
535 
536         self.btn_medir.setEnabled(False)
537         self.btn_guardar_resultados.setEnabled(False)
538 
539         self._lanzar_barrido(
540             frecuencias,
541             on_terminado=self._on_medicion_dut_terminada,
542             on_abortado=self._on_medicion_dut_abortada,
543         )
544         # Reconectamos punto_medido a un manejador especifico para DUT
545         # (ademas del log generico que ya se conecto en _lanzar_barrido)
546         self.hilo.punto_medido.connect(self._on_punto_dut)
547 
548     def _on_medicion_dut_abortada(self):
549         self._on_abortado_generico()
550         self.btn_medir.setEnabled(True)
551 
552     def _indice_calibracion_valido(self, i_1_based):
553         idx = i_1_based - 1
554         return self.cal_e00 is not None and 0 <= idx < len(self.cal_e00)
555 
556     def _on_punto_dut(self, i, n, f_rf, s11_crudo, f_batido):
557         # Si hay calibracion punto-a-punto disponible, corregimos en vivo.
558         if self._indice_calibracion_valido(i):
```

```
559             idx = i - 1
560             gamma = core.correccion_punto(
561                 s11_crudo, self.cal_e00[idx], self.cal_e11[idx],
                    ↪ self.cal_delta_e[idx]
562             )
563         else:
564             gamma = s11_crudo  # fallback: se corrige todo al final
565 
566         mod_db = 20 * np.log10(abs(gamma) + 1e-12)
567         fase_deg = np.degrees(np.angle(gamma))
568 
569         self.buf_freq.append(f_rf)
570         self.buf_mod_db.append(mod_db)
571         self.buf_fase_deg.append(fase_deg)
572         self.buf_gamma.append(gamma)
573 
574         self.linea_mod.set_data(self.buf_freq, self.buf_mod_db)
575         self.linea_fase.set_data(self.buf_freq, self.buf_fase_deg)
576         for ax in (self.ax_mod, self.ax_fase):
577             ax.relim()
578             ax.autoscale_view()
579         self.canvas_mf.draw_idle()
580 
581         self.linea_smith.set_data([g.real for g in self.buf_gamma],
582                                   [g.imag for g in self.buf_gamma])
583         self.canvas_smith.draw_idle()
584 
585     def _on_medicion_dut_terminada(self, freqs, gm_dut):
586         self.btn_abortar.setEnabled(False)
587         self.btn_medir.setEnabled(True)
588 
589         if len(freqs) != len(self.cal_e00):
590             QMessageBox.warning(
591                 self, "Advertencia de consistencia",
592                 "La cantidad de puntos medidos no coincide con la calibracion "
593                 "(revisa el rango de frecuencias). Se muestran los valores "
594                 "corregidos en vivo, pero puede haber un corrimiento de indice."
595             )
596 
597         n = min(len(freqs), len(self.cal_e00))
598         gamma_corregido = core.correccion(
599             gm_dut[:n], self.cal_e00[:n], self.cal_e11[:n],
                ↪ self.cal_delta_e[:n], n
600         )
601         mod_db, fase_deg = core.modulo_fase(gamma_corregido)
602 
603         self.ultima_medicion_freqs = freqs[:n]
604         self.ultima_medicion_gamma = gamma_corregido
605         self.ultima_medicion_mod_db = mod_db
606         self.ultima_medicion_fase_deg = fase_deg
607 
608         self.btn_guardar_resultados.setEnabled(True)
```

```
609             self._log(f"Medicion de DUT finalizada: {n} puntos corregidos.")
610
611         def _guardar_resultados(self):
612             if self.ultima_medicion_freqs is None:
613                 return
614             path, _ = QFileDialog.getSaveFileName(
615                 self, "Guardar resultados", "resultado_S11.csv", "CSV (*.csv)"
616             )
617             if not path:
618                 return
619             core.guardar_resultados_csv(
620                 path, self.ultima_medicion_freqs,
621                 self.ultima_medicion_mod_db, self.ultima_medicion_fase_deg,
622             )
623             self._log(f"Resultados guardados en {path}")
624
625         # ------------------------------------------------------------ Cierre --
626
627         def closeEvent(self, event):
628             if self.hilo is not None and self.hilo.isRunning():
629                 self.hilo.abortar()
630                 self.hilo.wait(2000)
631             if self.ser is not None:
632                 try:
633                     self.ser.close()
634                 except Exception:
635                     pass
636             event.accept()
637
638
639 def main():
640     app = QApplication(sys.argv)
641     app.setStyleSheet("""
642         QMainWindow, QWidget, QGroupBox {background-color: #fad7ea; }""")
643     ventana = MainWindow()
644     ventana.show()
645     sys.exit(app.exec())
646
647
648 if __name__ == "__main__":
649     main()
```

Apéndices

Bibliografía

[1] E. Topsakal, T. Karacolak, and E. C. Moreland. "Glucose-dependent dielectric properties of blood plasma." In: *Proceedings of the XXXth URSI General Assembly and Scientific Symposium* (2011). DOI: 10.1109/URSIGASS.2011.6051324.

[2] World Health Organization. *Global report on diabetes.* Sitio Web Oficial. 2016. URL: https://www.who.int/publications/i/item/9789241565257.

[3] World Health Organization. *Diabetes.* Sitio Web Oficial. Nov. 2024. URL: https://www.who.int/news-room/fact-sheets/detail/diabetes.

[4] Sitio Web Oficial. URL: https://www.freestylelibre.com.ar/.

[5] Tektronix Inc. *Vector Network Analysis: A Practical Introduction.* 2023. URL: https://download.tek.com/document/70W_60918_0_Tek_VNA_PR1.pdf?_gl=1*wh717q*_gcl_au*MjQyMjEyMTE5LjE3ODk0ODkzMjU.*_ga*NjkwMjAxNzcxLjE3ODk0ODkzMjU.*_ga_1HMYS1JH9M*czE3OTAxODY4MjkkbzIkZzAkdDE3OTAxODY4MzIkajU3JGwwJGgwJGRFXzVmVFRzOGMwbmFqaG8zX

[6] Hewlett Packard Inc. Rytting D. "Network analyzer error models and calibration methods". In: *HP* (1998).

[7] Urvashi Sengal. *Directional Couplers: Their Operation and Application.* Mini-Circuits. Oct. 2020. URL: https://blog.minicircuits.com/directional-couplers-their-operation-and-application/.

[8] Keysight Technologies. *778D Coaxial Dual-Directional Coupler, 100 MHz to 2 GHz.* URL: https://www.keysight.com/us/en/product/778D/coaxial-dualdirectional-coupler-100-mhz-to-2-ghz.html.

[9] Mini-Circuits. *ZABDC20-322H-S+ Coaxial Directional Coupler.* URL: https://www.minicircuits.com/WebStore/dashboard.html?model=ZABDC20-322H-S%2B.

[10] Altium. *Via Stitching and Via Shielding in PCB Design.* s. f. URL: https://www.altium.com/documentation/altium-designer/via-stitching-via-shielding-pcb.

[11] Altium. *Consideraciones para el diseño de PCB de alta velocidad: Prácticas recomendadas para la colocación y el enrutamiento de condensadores de desacoplo.* Dec. 2020. URL: https://resources.altium.com/es/p/high-speed-pcb-design-considerations-bypass-capacitor-place-and-route-tips.

[12] Analog Devices. *HMC601 75 dB logarithmic detector/controller.* https://www.analog.com/en/products/hmc601.html.

[13] Dimitris G. Manolakis and Vinay K. Ingle. *Applied Digital Signal Processing: Theory and Practice.* Cambridge, UK: Cambridge University Press, 2011. ISBN: 978-0-521-11002-0.

[14] Alan V. Oppenheim and Ronald W. Schafer. *Discrete-time signal processing, second edition.* Upper Saddle River, NJ: Prentice Hall, 1999. ISBN: 0-13-754920-2.

[15] Altium Limited. *Altium Designer.* Version 26.1.1. Licencia educativa. 2026. URL: https://www.altium.com.

[16] Analog Devices. *Sintetizador de RF ADF4351*. Datasheet. URL: https://www.analog.com/media/en/technical-documentation/data-sheets/ADF4351.pdf.

[17] Analog Devices. *ADIsimPLL*. Herramienta de diseño. Version 5.9. URL: https://www.analog.com/en/lp/resources/adisimpll.html.

[18] *ECS-2200X Series SMD Clock Oscillator Datasheet*. Cristal oscilador 10MHz. ECS Inc. International. URL: https://ecsxtal.com/store/pdf/ecs-2200x.pdf.

[19] *74LVC2G07: Dual Buffers with Open Drain Outputs*. DS35162. Datasheet. Diodes Incorporated. URL: https://www.diodes.com/assets/Datasheets/74LVC2G07.pdf.

[20] Mini-Circuits. *TC1-1-13M+ Surface Mount RF Transformer, 50Ω, 4.5 to 3000 MHz*. Rev. G. Mini-Circuits. 2023. URL: https://www.minicircuits.com/pdfs/TC1-1-13M+.pdf.

[21] *MCP6021/1R/2/3/4: Rail-to-Rail Input/Output, 10 MHz Op Amps*. DS20001685E. Microchip Technology Inc. 2017. URL: https://ww1.microchip.com/downloads/en/DeviceDoc/20001685E.pdf (visited on 10/28/2026).

[22] Texas Instruments. *UA78M 500-mA, Positive-Voltage Linear Regulator Datasheet*. Rev. V, Document SLVS059V. Texas Instruments Incorporated. Feb. 2025. URL: https://www.ti.com/lit/ds/slvs059v/slvs059v.pdf.